

From Python to Silicon

How every inference in the PyTorch-Spark platform is turned into bytes, JVM tasks, CUDA calls, PCIe transfers, warps, registers and machine instructions, measured on three environments (Windows 11 native, WSL2/Docker Linux, AWS g4dn.xlarge) on 26 Sep 2026.

Environment	Host	OS	Python / torch / CUDA	GPU	CPU ISA used
Windows 11 native	LAPTOP-OU17PK M2	Windows-11-10.0.26200-SP0	3.12.10 / 2.14.0+cu126 / 12.6	NVIDIA GeForce GTX 1650	AVX2
WSL2 / Docker (Linux)	adf6827a7a84	Linux-6.18.33.2-microsoft-standard-WSL	3.11.15 / 2.6.0+cu126 / 12.6	NVIDIA GeForce GTX 1650	AVX2
AWS g4dn.xlarge	ip-10-0-0-240	Linux-6.8.0-1060-aws-x86_64-with-glibc	3.11.16 / 2.6.0+cu126 / 12.6	Tesla T4	AVX512

How to read this document: chapters follow one inference request downward through the stack. Each chapter shows (1) what the layer does, (2) the exact artefact captured on your runs - a hex dump, a log line, a kernel table or an instruction listing - and (3) a line-by-line decoding of it. Chapter 12 then maps every benchmark test (Phases 1-10) to the low-level effect that dominated it. All sources are in `results/campaign_20260926/`; the generator is `benchmark/build_lowlevel_pdf.py`.

Contents

1. The stack in one picture
2. Bits: how a sample is stored
3. Spark driver <-> JVM (Py4J)
4. Shipping data to workers: pickle frames and Arrow IPC
5. Python worker processes and task metrics
6. PyTorch dispatch: from `aten::linear` to `cudaLaunchKernel`
7. The CPU path: x86 ISA, SIMD registers, oneDNN JIT
8. Crossing the bus: CUDA driver, launch latency, PCIe DMA
9. The GPU: SMs, warps, registers, occupancy
10. Every kernel your models launch
11. Machine code: Triton -> TTIR -> LLVM -> PTX -> SASS
12. Test results for all modes and what dominated each
13. Failures, root-caused to the byte
14. Reproduce / file index

1. The stack in one picture

A distributed-mode inference (Mode 1 / cluster_benchmark / submit_job) crosses every layer below, top to bottom, then the results travel back up. Single-GPU and hybrid modes skip the Spark layers (rows 2-5): the Python process talks to the CUDA driver directly. That one difference explains most of the throughput gaps in chapter 12.

Python driver	your code: run_benchmark.py / cluster_benchmark.py / submit_job.py
Py4J socket	text commands over localhost TCP to the JVM
Spark driver JVM	DAGScheduler -> stages -> tasks; TaskScheduler assigns to executors
RPC / Netty	task = serialized closure + partition bytes to executor JVM
Executor JVM	PythonRunner opens a socket to a python worker (fork on Linux, spawn on Windows)
Python worker	unpickle partition (or Arrow batches) -> numpy -> torch.from_numpy (zero-copy)
PyTorch / ATen	dispatcher: aten::linear -> aten::addmm -> CUDA kernel choice
cuBLAS / cuDNN	pick a pre-compiled SASS kernel (volta_sgemm_128x32..., winograd...)
CUDA runtime/driver	cudaMemcpyAsync / cudaLaunchKernel -> command in pushbuffer, doorbell write
PCIe	DMA of data (pageable->staging->device or pinned direct); doorbell MMIO write
GPU front-end	host interface + copy engines + GigaThread scheduler hand blocks to SMs
SM / warp	4 schedulers issue 1 instr/clk per warp of 32 threads; registers, shared mem, L1
L2 / DRAM	128-byte sectors over a 128-bit GDDR5/GDDR6 bus

What moves at each layer and in what unit:

Layer	Unit of work	Unit of data	Measured on your runs (see chapter)
Spark scheduler	job -> stage -> task (1 per partition)	partition	task deserialize / run / GC ms (5)
Python worker	one process per concurrent task	pickled row or Arrow record batch	34,478 B frame for 64x128 fp32 (4)
ATen	operator (aten::addmm)	tensor (ptr, sizes, strides, dtype)	dispatch chain us (6)
CUDA driver	API call -> pushbuffer command	kernel params (<=4 KB) / DMA descriptor	launch 14-86 us (8)
PCIe	TLP packets, 128b/130b coded lanes	256 B max payload per TLP	GB/s pinned vs pageable (8)
GPU	grid -> blocks -> warps of 32 threads	32-bit registers, 128 B cache lines	regs/thread, occupancy (9, 10)
SM core	one SASS instruction per warp per issue	128-bit instruction words	listing (11)

2. Bits: how a sample is stored

The EW-signal models take a float32 vector of 128 values per sample; images are float32 tensors of 3x224x224; detection inputs 3x640x640. Everything below is from the real generator (data/signal_generator.py) on your run.

Batch of 4 EW signals: shape [4, 128], dtype torch.float32, stride [128, 1] elements, 2048 bytes, element size 4 B. data_ptr % 64 = 16 (not 64-byte aligned - numpy allocator; PyTorch-allocated tensors are 64 B aligned on CPU and 256 B+ on GPU).

2.1 IEEE-754 single precision, decoded

A float32 is 32 bits: bit 31 = sign, bits 30-23 = biased exponent (bias 127), bits 22-0 = fraction with an implicit leading 1. value = $(-1)^s \times 1.f \times 2^{(e-127)}$. fp16 keeps 5 exponent / 10 fraction bits; bf16 keeps fp32's 8 exponent bits but only 7 fraction bits (so it rounds more coarsely but never overflows where fp32 would not).

value	hex (memory order reversed)	s	exponent (bin / dec)	fraction (23 bits)	fp16	bf16
+0.10284512	0x3dd2a076	0	01111011 / 123	1010010101000001110110	+0.102844	+0.103027
+0.15720458	0x3e20fa3d	0	01111100 / 124	01000001111101000111101	+0.157227	+0.157227
+0.12035759	0x3df67e0a	0	01111011 / 123	11101100111111000001010	+0.120361	+0.120117
+0.10617970	0x3dd974be	0	01111011 / 123	10110010111010010111110	+0.106201	+0.105957
+0.10612669	0x3dd958f3	0	01111011 / 123	10110010101100011110011	+0.106140	+0.105957
+0.11295056	0x3de752a0	0	01111011 / 123	11001110101001010100000	+0.112976	+0.112793

Worked example: 0x3dd2a076 -> sign 0, exponent 123 - 127 = -4, fraction 0x52a076/2²³ = 0.64552188, so value = 1.52a076h x 2⁻⁴ = 0.10284512. x86 and NVIDIA GPUs are little-endian, so in memory the four bytes appear lowest first: 76 a0 d2 3d.

2.2 The raw bytes of the tensor

First 64 bytes of the storage as they sit in RAM (and later on the PCIe bus and in GPU DRAM - the bytes are never converted, only copied):

```
0000 76 a0 d2 3d 3d fa 20 3e 0a 7e f6 3d be 74 d9 3d v...=, >.,.=.t.=
0010 f3 58 d9 3d a0 52 e7 3d 0a 14 e3 3d 73 fd f6 3d .X.=.R.=...S.=
0020 39 e9 f7 3d 50 09 f3 3d be 9b 0f 3e 0d 89 10 3e 9..=P..=>...>
0030 a3 36 9b 3d 8f aa ff 3d 21 36 e1 3d 83 77 e1 3d .6.=...!=6.=.w.=
```

2.3 Layout: strides

An image batch [1,3,224,224] in default NCHW layout has strides (150528, 50176, 224, 1): moving one channel skips 50,176 floats, one row 224, one pixel 1. The same tensor in channels_last (NHWC) has strides (150528, 1, 672, 3): the 3 colour values of a pixel are adjacent. cuDNN and oneDNN reorder into their own blocked layouts (e.g. oneDNN's aBcd8b / aBcd16b in chapter 7 = channels grouped in blocks of 8 or 16 to match one SIMD register).

3. Spark driver <-> JVM (Py4J)

PySpark is a thin client. The Python driver starts a JVM (spark-submit) and talks to it through Py4J: a localhost TCP socket carrying line-oriented text commands. `sc.parallelize(...)`, `rdd.mapPartitions(...).collect()` each become a command such as `c\no12\ncollectAndServe\nro34\ne\n` (call, object id, method, argument refs, end); the JVM runs it by reflection and replies `yro35` (success, object reference). Large data does not go through Py4J - `collect()` results come back over a separate authenticated local socket, and `parallelize()` writes the pickled data to a temp file the JVM reads.

	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
PySpark version	4.2.0	3.5.1	3.5.1
master URL	local[2]	local[2]	spark://10.0.0.240:7077
driver Python PID	24956	652	4473
driver JVM PID	4416	694	4493
Py4J gateway port (JVM listens)	52534	42573	32881
JVM	17.0.20.1	17.0.20	17.0.20.1
driver JVM max heap (MB)	1024	1024	1024

The ephemeral port changes every run; the JVM PID is the parent of every Python worker on Windows (next chapter), because there the JVM spawns workers itself.

4. Shipping data to workers: pickle frames and Arrow IPC

4.1 RDD path (Mode 1 / cluster_engine rdd): framed pickle

For RDD operations the executor JVM streams each partition to the Python worker over a local socket as a sequence of frames: a 4-byte big-endian length followed by that many bytes of pickle (protocol 5), batched 1024 records per frame by `BatchedSerializer`; a length of -1 (0xFFFFFFFF) ends the data section. The closure itself (your `mapPartitions` function and everything it references - in this platform the serialized model state_dicts) is sent the same way, cloudpickled, once per task.

Captured: one partition of 64 EW-signal rows (64 x 128 x 4 = 32,768 raw bytes) serializes to 34,478 bytes on the wire (5.2% overhead):

0000	00 00 86 aa 80 05 95 9f 86 00 00 00 00 00 00 5d	]
0010	94 28 8c 13 6e 75 6d 70 79 2e 5f 63 6f 72 65 2e	...numpy._core.
0020	6e 75 6d 65 72 69 63 94 8c 0b 5f 66 72 6f 6d 62	numeric..._fromb
0030	75 66 66 65 72 94 93 94 28 96 00 02 00 00 00 00	uffer...(.....
0040	00 00 6a 0c b9 be 9b 37 74 be 62 8d ea be 13 41	..j....7t.b....A
0050	20 bf bd 89 c3 be e1 af d3 be 11 1e 54 bf 35 90	T.5.

bytes	meaning
00 00 86 aa	frame length, big-endian int32 = 0x86aa = 34,474 bytes of pickle follow
80 05	PROTO 5 - pickle protocol version 5
95 9f 86 00 00 00 00 00 00	FRAME opcode + 8-byte little-endian frame size (0x869f)
5d 94 28	EMPTY_LIST, MEMOIZE, MARK - start of the batch list
8c 13 'numpy._core.numeric'	SHORT_BINUNICODE: module name (19 chars)
8c 0b '_frombuffer' 94 93	SHORT_BINUNICODE function name, MEMOIZE, STACK_GLOBAL -> numpy._core.numeric._frombuffer
28 96 00 02 00 00 00 00 00	MARK, BYTEARRAY8 with 8-byte length 0x200 = 512 bytes = 128 floats (one row)
6a 0c b9 be ...	the 128 little-endian float32 values themselves (e.g. be b9 0c 6a = -0.3614...)

So each row costs ~27 bytes of pickle metadata (module/function names are memoized after the first row) plus a Python object allocation on each side - fine for 5K signals, visible for images (3x224x224 = 602,112 bytes per sample).

4.2 pandas_udf path (engine=udf): Arrow IPC

With `engine=udf` the JVM converts rows to Apache Arrow record batches (columnar, `spark.sql.execution.arrow.maxRecordsPerBatch` rows each) and writes them in the Arrow IPC streaming format; the worker maps them to pandas without per-row Python objects.

```

validity bitmap = 0d (bits LSB-first: 1,0,1,1 -> 0b1101 = 0x0d)
data buffer      = 0000c03f00000000000010c000004040 (4 x fp32 little-endian, null slot is garbage/zero)
IPC stream 296 bytes: 0xFFFFFFFF continuation, int32 metadata length, flatbuffer Schema, then RecordBatch message
+ 8-byte-aligned body buffers:
0000 ff ff ff ff 70 00 00 00 10 00 00 00 00 00 0a 00 ....p.....
0010 0c 00 06 00 05 00 08 00 0a 00 00 00 00 01 04 00 .....
0020 0c 00 00 00 08 00 08 00 00 00 04 00 08 00 00 00 .....
0030 04 00 00 00 01 00 00 00 14 00 00 00 10 00 14 00 .....
0040 08 00 06 00 07 00 0c 00 00 00 10 00 10 00 00 00 .....
0050 00 00 01 03 10 00 00 00 18 00 00 00 04 00 00 00 .....

```

bytes	meaning		
ff ff ff ff	continuation marker (IPC format >= 0.15)		
70 00 00 00	metadata length = 112 bytes of FlatBuffer follow		
10 00 00 00 ... 0a 00 0c 00 ...	FlatBuffer Message{version=V5, header=Schema}: a vtable of little-endian uint16 field offsets, then the table		
... 01 03 ...	Type = FloatingPoint, precision = SINGLE (the float32 column 'x')		
(later) RecordBatch message	length=4, nodes=[{length 4, null_count 1}], buffers=[validity @0 len 1, data @8 len 16]		
0d 00 00 00 00 00 00 00	validity bitmap, LSB first: 1 0 1 1 = 0x0d (element 1 is null), padded to 8 bytes		
00 00 c0 3f 00 00 00 00 00 00 10 c0 00 00 40 40	1.5 (null slot) -2.25 3.0 as little-endian IEEE-754		
	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
200k rows x2, pandas_udf (Arrow)	5.85 s	3.30 s	1.94 s
200k rows x2, RDD map (pickle)	14.24 s	3.83 s	1.38 s

On Linux the pickle path wins for this trivial function (workers are forked and reused, and Arrow adds JVM-side conversion); on Windows every task pays a full Python process start, so the path with fewer round trips (Arrow) wins. With real models the per-sample cost of pickling large arrays dominates and Arrow/UDF returns per-sample predictions - see eng_* tests in chapter 12.

5. Python worker processes and task metrics

Each Spark task that runs Python code needs a Python process. On Linux the executor JVM starts one python `-m pyspark.daemon` which fork()s a worker per concurrent task (copy-on-write: the new process shares the parent's already-imported modules). With `spark.python.worker.reuse=true` (set by this platform) the worker survives for the next task. Windows has no fork(), so the JVM launches python `-m pyspark.worker` directly and every worker pays a full interpreter start plus 'import torch' (~1-3 s).

Windows 11 native - the 4 tasks of a probe job ran in:

pid	ppid (parent)	host	rows	compute us
25920	4416	LAPTOP-OU17PKM2	5000	2583
32300	4416	LAPTOP-OU17PKM2	5000	2136
28868	4416	LAPTOP-OU17PKM2	5000	3055
12192	4416	LAPTOP-OU17PKM2	5000	2906

Parent PID(s) [4416]; driver JVM PID 4416. Parent is the JVM itself - workers are spawned, not forked.

WSL2 / Docker (Linux) - the 4 tasks of a probe job ran in:

pid	ppid (parent)	host	rows	compute us
794	791	adf6827a7a84	5000	601
796	791	adf6827a7a84	5000	599
796	791	adf6827a7a84	5000	288
794	791	adf6827a7a84	5000	472

Parent PID(s) [791]; driver JVM PID 694. All workers share one parent = forked from pyspark.daemon.

AWS g4dn.xlarge - the 4 tasks of a probe job ran in:

pid	ppid (parent)	host	rows	compute us
3596	3592	ip-10-0-0-240	5000	544
3606	3592	ip-10-0-0-240	5000	530
3595	3592	ip-10-0-0-240	5000	670
3603	3592	ip-10-0-0-240	5000	484

Parent PID(s) [3592]; driver JVM PID 4493. All workers share one parent = forked from pyspark.daemon.

5.1 Stage metrics (Spark REST API) of the same probe jobs

executorRunTime is wall time of the task thread in the executor JVM; executorCpuTime is CPU time of that JVM thread only. Python work happens in another process, so it shows up as run time without CPU time - the gap is time the JVM spent blocked on the Python socket (and on the GPU, for inference tasks).

Windows 11 native

stage	tasks	run ms	JVM cpu ms	deserialize ms	result ser ms	GC ms	records in
0	4	31357	516	320	27	36	0
1	4	967	703	159	17	14	200000
2	4	0	0	0	0	0	0
3	1	196	203	28	13	6	0
4	4	27207	1047	99	0	14	200000

WSL2 / Docker (Linux)

stage	tasks	run ms	JVM cpu ms	deserialize ms	result ser ms	GC ms	records in
0	4	5896	195	178	40	34	0
1	4	488	297	82	13	0	200000
2	4	0	0	0	0	0	0
3	1	119	116	21	4	0	0
4	4	6333	191	132	12	30	200000

AWS g4dn.xlarge

stage	tasks	run ms	JVM cpu ms	deserialize ms	result ser ms	GC ms	records in
0	4	3329	115	1463	19	40	0
1	4	1558	438	643	1	32	200000
2	4	0	0	0	0	0	0
3	1	126	90	48	1	0	0
4	4	3824	61	542	17	354	200000

5.2 One real inference task, from the Spark event log

AWS g4dn.xlarge, application ClusterBench_gpu_only_1000 (app-20260926115836-0016), 2 tasks. The event log is a JSON line per listener event: ApplicationEnd, ApplicationStart, BlockManagerAdded, EnvironmentUpdate, ExecutorAdded, JobEnd, JobStart, LogStart, ResourceProfileAdded, StageCompleted, StageSubmitted, TaskEnd, TaskStart.

task	executor@host	locality	wall ms	deser ms	deser CPU ms	run ms	run CPU ms	GC ms	result B
1	0@10.0.0.240	PROCESS_LOCAL	8337	290	132	6181	19	40	1875
0	0@10.0.0.240	PROCESS_LOCAL	8772	792	363	6211	74	52	1875

Read it as: deserialize = JVM unpacks the task closure (includes the serialized model weights shipped by the driver); run = the JVM waits while the Python worker loads 10 models, copies data host->device, runs kernels and returns counts; run CPU (JVM) stays tiny because the JVM thread is just blocked on a socket read. Result size is small because this platform returns counts/timings, not predictions, from each partition.

6. PyTorch dispatch: from aten::linear to cudaLaunchKernel

Calling model(x) runs Python forward() code which calls torch operators. Each operator goes through the ATen dispatcher (a table keyed by device/dtype/autograd 'dispatch keys'), lands in the CUDA implementation, which asks cuBLAS/cuDNN for a kernel, fills a small parameter block, and calls cudaLaunchKernel. The CUPTI 'correlation id' links the CPU-side launch call to the GPU-side kernel. Below: the first Linear layer of ew_classifier and the first Conv of ResNet18, with host-side timestamps relative to the operator start.

Windows 11 native

```
Dispatch chain inside the first aten::linear (host timeline):
+ 0.0us 2313.5us [cpu_op] aten::linear
+ 400.7us 490.0us [cpu_op] aten::t
+ 419.8us 462.3us [cpu_op] aten::transpose
+ 863.9us 14.5us [cpu_op] aten::as_strided
+ 903.8us 1400.8us [cpu_op] aten::addmm
+ 1019.2us 1261.5us [cuda_runtime] cudaLaunchKernel
-> GPU kernels launched by it (matched by CUPTI correlation id): ['volta_sgemm_128x32_sliced1x4_tn']
```

Windows 11 native

```
Dispatch chain inside the first aten::conv2d (host timeline):
+ 0.0us 967.3us [cpu_op] aten::conv2d
+ 19.5us 944.3us [cpu_op] aten::convolution
+ 38.5us 919.8us [cpu_op] aten::_convolution
+ 498.8us 442.0us [cpu_op] aten::cudnn_convolution
+ 631.9us 121.7us [cuda_runtime] cudaLaunchKernel
+ 755.7us 0.8us [cuda_runtime] cudaPeekAtLastError
+ 763.5us 1.6us [cuda_runtime] cudaMemsetAsync
+ 766.8us 140.2us [cuda_runtime] cudaLaunchKernel
+ 909.5us 0.4us [cuda_runtime] cudaPeekAtLastError
-> GPU kernels launched by it (matched by CUPTI correlation id): ['_ZN14cask__5x_cudnn20computeOffsetsKernelILb0ELb0EEEEvN
S_20ComputeOffse', '_5x_cudnn_volta_scudnn_128x64_relu_xregs_large_nn_v1']
```

WSL2 / Docker (Linux)

```
Dispatch chain inside the first aten::linear (host timeline):
+ 0.0us 1771.7us [cpu_op] aten::linear
+ 59.8us 693.8us [cpu_op] aten::t
+ 126.7us 617.4us [cpu_op] aten::transpose
+ 728.7us 10.4us [cpu_op] aten::as_strided
+ 808.4us 877.8us [cpu_op] aten::addmm
+ 948.7us 21.5us [cpu_op] aten::empty
+ 1197.4us 463.5us [cuda_runtime] cudaLaunchKernel
-> GPU kernels launched by it (matched by CUPTI correlation id): ['volta_sgemm_128x32_sliced1x4_tn']
```

WSL2 / Docker (Linux)

```
Dispatch chain inside the first aten::conv2d (host timeline):
+ 0.0us 3002.0us [cpu_op] aten::conv2d
+ 207.3us 2790.5us [cpu_op] aten::convolution
+ 814.8us 2125.5us [cpu_op] aten::_convolution
+ 1313.8us 1336.5us [cpu_op] aten::cudnn_convolution
+ 1438.3us 1030.0us [cuda_runtime] cudaLaunchKernel
+ 2473.0us 1.6us [cuda_runtime] cudaPeekAtLastError
+ 2484.1us 1.8us [cuda_runtime] cudaMemsetAsync
+ 2487.6us 103.8us [cuda_runtime] cudaLaunchKernel
+ 2592.3us 0.4us [cuda_runtime] cudaPeekAtLastError
-> GPU kernels launched by it (matched by CUPTI correlation id): ['void cask__5x_cudnn::computeOffsetsKernel<false, false
>(cask__5x_cudnn', '_5x_cudnn_volta_scudnn_128x64_relu_medium_nn_v1']
```

AWS g4dn.xlarge

```
Dispatch chain inside the first aten::linear (host timeline):
+ 0.0us 210.7us [cpu_op] aten::linear
+ 6.1us 20.7us [cpu_op] aten::t
+ 13.6us 9.7us [cpu_op] aten::transpose
+ 19.4us 3.4us [cpu_op] aten::as_strided
+ 28.8us 180.2us [cpu_op] aten::addmm
+ 60.0us 6.4us [cpu_op] aten::empty
+ 117.5us 52.8us [cuda_runtime] cudaOccupancyMaxActiveBlocksPerMultiprocessor
+ 174.2us 28.5us [cuda_runtime] cudaLaunchKernel
-> GPU kernels launched by it (matched by CUPTI correlation id): ['volta_sgemm_32x32_sliced1x4_tn']
```

AWS g4dn.xlarge

```
Dispatch chain inside the first aten::conv2d (host timeline):
+ 0.0us 110.3us [cpu_op] aten::conv2d
+ 3.5us 106.4us [cpu_op] aten::convolution
+ 9.0us 99.8us [cpu_op] aten::_convolution
+ 23.3us 83.8us [cpu_op] aten::cudnn_convolution
+ 72.1us 19.4us [cuda_runtime] cudaLaunchKernel
+ 92.1us 0.4us [cuda_runtime] cudaPeekAtLastError
+ 94.2us 0.3us [cuda_runtime] cudaMemsetAsync
+ 94.9us 6.1us [cuda_runtime] cudaLaunchKernel
+ 101.5us 0.1us [cuda_runtime] cudaPeekAtLastError
-> GPU kernels launched by it (matched by CUPTI correlation id): ['void cask__5x_cudnn::computeOffsetsKernel<false, false>(cask__5x_cudnn', '_5x_cudnn_volta_scudnn_128x64_relu_medium_nn_v1']
```

Note the dispatch overhead is paid on the CPU for every operator, every batch: ~0.2 ms per Linear on the AWS Xeon, ~2.3 ms on the Windows laptop (profiler overhead included). With a small MLP on 256 samples this CPU overhead is larger than the GPU work - which is why the tiny signal models reach only a few thousand samples/s even on a GPU, and why batching matters (Phase 4/8).

7. The CPU path: x86 ISA, SIMD registers, oneDNN JIT

In `cpu_only` mode, and for any model the hybrid planner places on CPU, the same `aten::linear/aten::conv2d` calls go to oneDNN (for conv) and MKL/oneDNN GEMM (for linear). These libraries contain several copies of each kernel for different instruction sets and pick one at run time using `CPUID`; oneDNN additionally JIT-generates x86 machine code (Xbyak) specialised to the exact shape.

		Windows 11 native		WSL2 / Docker (Linux)		AWS g4dn.xlarge	
CPU		Intel64 Family 6 Model 158 Stepping 10, GenuineIntel		x86_64		x86_64	
logical CPUs		8		8		4	
ATen SIMD dispatch		AVX2		AVX2		AVX512	
intra-op threads		4		4		2	
SIMD flags (Linux)		(Windows: not exposed)		sse, sse2, ssse3, fma, sse4_1, sse4_2, avx, f16c, avx2		sse, sse2, ssse3, fma, sse4_1, sse4_2, avx, f16c, avx2, avx512f, avx512dq, avx512cd, avx512bw, avx512vl, avx512_vnni	
ISA	registers	width	fp32 per register	FMA FLOP / instr	where		
SSE4.2	16 x XMM	128 bit	4	- (no FMA)	baseline		
AVX2 + FMA3	16 x YMM	256 bit	8	16	i5-9300H laptop (Windows & WSL2)		
AVX-512 + VNNI	32 x ZMM + 8 mask k0-k7	512 bit	16	32 (int8 VNNI: 64 MAC)	Xeon 8259CL on AWS		

7.1 oneDNN verbose log, decoded

Windows 11 native

```
onednn_verbose,v1,primitive,exec,cpu,reorder,jit:uni,undef,src:f32::blocked:abcd::f0 dst:f32::blocked:Acdb8a::f0,attr-scratchpad:user,,64x3x7x7,5.3787
onednn_verbose,v1,primitive,exec,cpu,convolution,jit:avx2,forward_training,src:f32:a:blocked:abcd::f0 wei:f32:a:blocked:Acdb8a::f0 bia:f32:a:blocked:a::f0 dst:f32:a:blocked:aBcd8b::f0,attr-scratchpad:user,alg:convolution_direct,mb8_ic3oc64_ih224oh112kh7sh2dh0ph3_iw224ow112kw7sw2dw0pw3,29.9781
onednn_verbose,v1,primitive,exec,cpu,reorder,jit:blk,undef,src:f32::blocked:aBcd8b::f0 dst:f32::blocked:abcd::f0,attr-scratchpad:user,,8x64x112x112,23.9946
```

WSL2 / Docker (Linux)

```
onednn_verbose,primitive,exec,cpu,reorder,jit:uni,undef,src_f32::blocked:abcd::f0 dst_f32::blocked:Acdb8a::f0,attr-scratchpad:user,,64x3x7x7,1.552
onednn_verbose,primitive,exec,cpu,convolution,jit:avx2,forward_training,src_f32:a:blocked:abcd::f0 wei_f32:a:blocked:Acdb8a::f0 bia_f32:a:blocked:a::f0 dst_f32:a:blocked:aBcd8b::f0,attr-scratchpad:user,alg:convolution_direct,mb8_ic3oc64_ih224oh112kh7sh2dh0ph3_iw224ow112kw7sw2dw0pw3,62.8162
onednn_verbose,primitive,exec,cpu,reorder,jit:blk,undef,src_f32::blocked:aBcd8b::f0 dst_f32::blocked:abcd::f0,attr-scratchpad:user,,8x64x112x112,33.198
```

AWS g4dn.xlarge

<pre> onednn_verbose,primitive,exec,cpu,reorder,jit:uni,undef,src_f32::blocked:abcd::f0 dst_f32::blocked:Acdb16a::f0,attr-scratchpad:user,,64x3x7x7,0.147949 onednn_verbose,primitive,exec,cpu,convolution,jit:avx512_core,forward_training,src_f32:a:blocked:abcd::f0 wei_f32:a:blocked:Acdb16a::f0 bia_f32:a:blocked:a::f0 dst_f32:a:blocked:aBcd16b::f0,attr-scratchpad:user,alg:convolution_direct,mb8_ic3oc64_ih224oh112kh7sh2dh0ph3_iw224ow112kw7sw2dw0pw3,17.2219 onednn_verbose,primitive,exec,cpu,reorder,jit:blk,undef,src_f32::blocked:aBcd16b::f0 dst_f32::blocked:abcd::f0,attr-scratchpad:user,,8x64x112x112,9.06885 </pre>	
field	meaning
primitive,exec,cpu	a primitive executed on the CPU engine
convolution,jit:avx2 / jit:avx512_core	implementation: JIT-generated code for the AVX2 or AVX-512 ISA (Xbyak emits the machine code at run time)
reorder,jit:uni / jit:blk	layout conversion kernels (weights to blocked layout, output back to plain)
src:f32::blocked:abcd	source tensor fp32 in plain NCHW (a=N, b=C, c=H, d=W)
wei ... Acdb8a / Acdb16a	weights blocked by 8 or 16 output channels innermost = exactly one YMM (8 fp32) or ZMM (16 fp32) register
dst ... aBcd8b / aBcd16b	output channels in blocks of 8/16 so one vector FMA updates 8/16 output channels at once
64x3x7x7, 8x64x112x112	problem shape (ResNet stem conv 7x7, 64 filters) - last number = execution time in ms
The block size in the layout tag (8 on your laptop, 16 on AWS) is the SIMD width showing through: the same PyTorch call produces a different memory layout and different machine code on each CPU.	

7.2 The inner loop of every Linear layer, as x86 machine instructions

A Linear layer is a matrix multiply, whose core is a dot product $s += a[i]*b[i]$. Compiled with gcc -O3 -mavx2 -mfma -ffast-math, the loop becomes vectorised FMA instructions operating on YMM registers:

instruction	what the CPU does
endbr64	Intel CET landing pad: marks a legal indirect-branch target (security, no-op otherwise).
movq %rsi, %rcx	Integer move (copy n, pointers).
testl %edx, %edx	Compare -> sets FLAGS.
jle .L9	Conditional jump on FLAGS (loop back-edge / tail handling).
leal -1(%rdx), %eax	Integer loop-counter / pointer arithmetic.
cmpl \$6, %eax	Compare -> sets FLAGS.
jbe .L10	Conditional jump on FLAGS (loop back-edge / tail handling).
movl %edx, %esi	Integer move (copy n, pointers).
xorl %eax, %eax	xor reg,reg = zero the register (loop index i = 0).
vxorps %xmm0, %xmm0, %xmm0	$x \wedge x = 0$: the idiom for zeroing an accumulator register.
shrl \$3, %esi	Integer loop-counter / pointer arithmetic.
salq \$5, %rsi	Shift left: n/8 iterations x 32 bytes = byte length of the vector part.
vmovups (%rdi,%rax), %ymm3	Vector load/store (u = unaligned allowed).
vfmadd231ps (%rcx,%rax), %ymm3, %ymm0	FMA3 packed single: 8 (ymm) or 16 (zmm) fp32 multiply-adds in one instruction, one rounding.
addq \$32, %rax	Integer loop-counter / pointer arithmetic.
cmpq %rsi, %rax	Compare -> sets FLAGS.
jne .L4	Conditional jump on FLAGS (loop back-edge / tail handling).
vextractf128 \$0x1, %ymm0, %xmm1	Horizontal reduction: fold the 8 lanes of the accumulator into 1 float.
movl %edx, %eax	Integer move (copy n, pointers).
vaddps %xmm0, %xmm1, %xmm0	Packed add (horizontal reduction of partial sums).
andl \$-8, %eax	Integer loop-counter / pointer arithmetic.
movl %eax, %esi	Integer move (copy n, pointers).
vmovhlps %xmm0, %xmm0, %xmm1	Horizontal reduction: fold the 8 lanes of the accumulator into 1 float.
vaddps %xmm0, %xmm1, %xmm1	Packed add (horizontal reduction of partial sums).
vshufps \$85, %xmm1, %xmm1, %xmm0	Horizontal reduction: fold the 8 lanes of the accumulator into 1 float.
vaddps %xmm1, %xmm0, %xmm0	Packed add (horizontal reduction of partial sums).
cmpl %eax, %edx	Compare -> sets FLAGS.
je .L14	Conditional jump on FLAGS (loop back-edge / tail handling).

instruction	what the CPU does
vzeroupper	Clear upper YMM halves before returning to SSE code (avoids AVX-SSE transition penalty).
movl %edx, %r8d	Integer move (copy n, pointers).
subl %eax, %r8d	Integer loop-counter / pointer arithmetic.
leal -1(%r8), %r9d	Integer loop-counter / pointer arithmetic.
cmpl \$2, %r9d	Compare -> sets FLAGS.
jbe .L7	Conditional jump on FLAGS (loop back-edge / tail handling).
vmovups (%rdi,%rax,4), %xmm4	Vector load/store (u = unaligned allowed).
vmulps (%rcx,%rax,4), %xmm4, %xmm1	Packed multiply.
movl %r8d, %eax	Integer move (copy n, pointers).
andl \$-4, %eax	Integer loop-counter / pointer arithmetic.
addl %eax, %esi	Integer loop-counter / pointer arithmetic.
vmovhlps %xmm1, %xmm1, %xmm2	Horizontal reduction: fold the 8 lanes of the accumulator into 1 float.
vaddps %xmm1, %xmm2, %xmm2	Packed add (horizontal reduction of partial sums).
vshufps \$85, %xmm2, %xmm2, %xmm1	Horizontal reduction: fold the 8 lanes of the accumulator into 1 float.
vaddps %xmm2, %xmm1, %xmm1	Packed add (horizontal reduction of partial sums).
vaddss %xmm1, %xmm0, %xmm0	
cmpl %eax, %r8d	Compare -> sets FLAGS.
je .L1	Conditional jump on FLAGS (loop back-edge / tail handling).
The hot loop is the four instructions vmovups / vmadd231ps / addq \$32 / jne: each iteration loads 8 floats of a (32 bytes, one YMM), multiplies them by 8 floats of b straight from memory and accumulates into ymm0 - 16 FLOP per iteration. After the loop the 8 partial sums in ymm0 are folded (vextractf128, vmovhlps, vshufps + vaddps) into one float, then a 4-wide (xmm) and scalar tail handle n % 8. MKL/oneDNN's real GEMM kernels are the same idea unrolled over a register tile: 12+ YMM accumulators, each loaded value reused many times.	

8. Crossing the bus: CUDA driver, launch latency, PCIe DMA

A kernel launch does not 'call' the GPU. The CUDA driver writes a command (method + kernel parameters, grid/block sizes, shared-memory size, the address of the SASS code) into a pushbuffer - a ring buffer in pinned host memory the GPU can read - and then writes a doorbell register on the GPU through a PCIe memory-mapped (MMIO) write. The GPU's host interface fetches the commands by DMA over PCIe and hands blocks to SMs. On Windows the WDDM display-driver model inserts the OS graphics kernel (dxgkrnl) and a scheduling queue between the CUDA user-mode driver and the hardware; WSL2 goes through a paravirtualized /dev/dxg into the same Windows host driver; Linux on AWS uses the native NVIDIA kernel driver directly.

	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
kernel launch, CPU enqueue (us)	86.4	245.6	13.8
launch + sync round trip (us)	89.7	234.3	17.6
PCIe link	Gen3 x16	Gen3 x16	Gen3 x8
PCIe theoretical per direction	15.76 GB/s	15.76 GB/s	7.88 GB/s

PCIe arithmetic: Gen3 = 8 GT/s per lane with 128b/130b line coding -> $8 \times 128/130 / 8 = 0.985$ GB/s per lane per direction. x16 = 15.75 GB/s, x8 = 7.88 GB/s. Real transfers also carry TLP headers (~20-24 B per <=256 B payload), so ~80-85% of line rate is the practical ceiling.

8.1 Host <-> device copy bandwidth

Windows 11 native (GB/s)

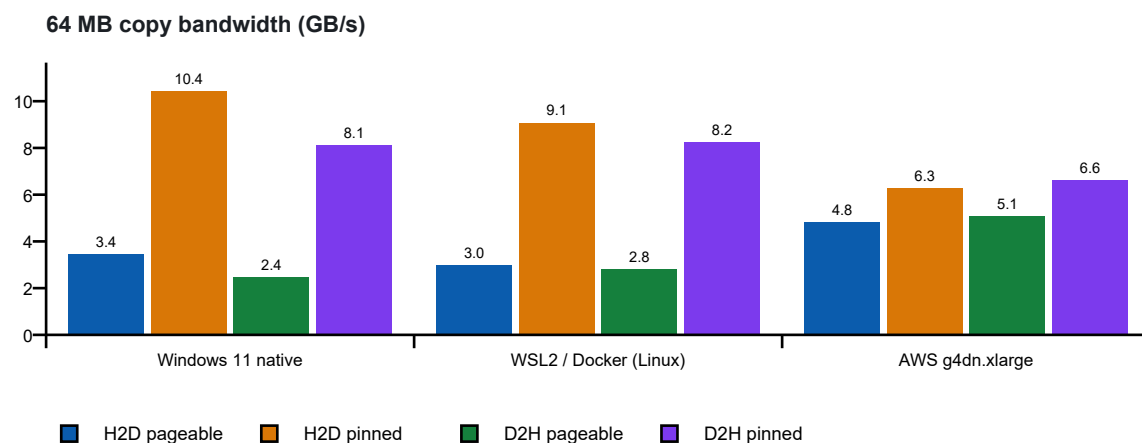
size	H2D pageable	H2D pinned	D2H pageable	D2H pinned
0.004 MB	0.07	0.08	0.09	0.11
0.064 MB	0.77	1.36	0.77	1.37
1 MB	4.91	9.22	2.28	10.66
4 MB	3.65	11.28	3.54	7.51
16 MB	2.54	10.46	2.71	8.58
64 MB	3.44	10.40	2.44	8.12
256 MB	2.77	8.32	2.37	8.54

WSL2 / Docker (Linux) (GB/s)

size	H2D pageable	H2D pinned	D2H pageable	D2H pinned
0.004 MB	0.06	0.12	0.06	0.20
0.064 MB	1.61	3.16	0.83	3.33
1 MB	3.08	10.12	1.63	8.41
4 MB	3.73	6.75	2.18	8.74
16 MB	1.98	8.29	1.70	4.67
64 MB	2.99	9.07	2.80	8.23
256 MB	2.27	9.40	2.58	6.76

AWS g4dn.xlarge (GB/s)

size	H2D pageable	H2D pinned	D2H pageable	D2H pinned
0.004 MB	0.44	0.50	0.34	0.63
0.064 MB	3.01	5.31	1.91	5.79
1 MB	4.64	6.19	2.77	6.53
4 MB	5.03	6.25	4.20	6.58
16 MB	5.20	6.26	4.89	6.60
64 MB	4.82	6.27	5.09	6.61
256 MB	4.79	6.27	5.14	6.61



Pageable memory: the driver cannot DMA from pages the OS may move or swap, so it copies your buffer (with the CPU) into a pinned staging buffer in chunks and DMA's those - two copies, CPU-bound. Pinned (page-locked) memory is DMA'd directly by the GPU copy engine. In this platform the inputs are created by numpy (pageable) and moved with `.to(device)`, so every inference pays the pageable rate (visible in the kernel traces as 'Memcpy HtoD (Pageable -> Device)').

8.2 Device memory and compute ceilings

	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
DRAM, measured D2D copy (GB/s)	95.9	109.4	231.0
DRAM spec (GB/s)	128 (GDDR5 8 Gbps x 128-bit)	128 (GDDR5 8 Gbps x 128-bit)	320 (GDDR6, 256-bit eff.)
FP32 peak (TFLOPS) = SMs x 64 x 2 x clock	4.30	4.30	8.14
SGEMM 2048^3 fp32 measured	1.86	2.60	6.95
HGEMM 2048^3 fp16 measured	0.41	0.40	43.89

Why fp16 is 4.5x slower than fp32 on the GTX 1650 but 6x faster on the T4: both are Turing (sm_75), but the TU117 chip in the GTX 1650 has no Tensor Cores, so fp16 matrix multiply falls back to generic kernels; the T4 (TU104) has 320 Tensor Cores that execute HMMA.1688 instructions (a 16x8x8 fp16 matrix multiply-accumulate per warp per instruction). Note on DRAM: the tracer's naive formula (clock x 2 x bus width) is right for GDDR5 but undercounts GDDR6, which moves 2 bits per pin per reported clock edge pair; the T4's measured 231 GB/s (72% of 320) proves the spec figure.

9. The GPU: SMs, warps, registers, occupancy

property	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
GPU	NVIDIA GeForce GTX 1650	NVIDIA GeForce GTX 1650	Tesla T4
compute capability major	7	7	7
minor	5	5	5
SMs	16	16	40
threads per warp	32	32	32
max threads per SM	1024	1024	1024
32-bit registers per SM	65536	65536	65536
shared memory per SM (B)	65536	-	-
L2 cache (B)	1048576	1048576	4194304
memory bus (bits)	128	-	-
VRAM (B)	4294705152	4294705152	15637086208
FP32 lanes ('CUDA cores')	1024	1024	2560
max SM clock (MHz)	2100	2100	1590

A Turing SM has 4 processing blocks. Each has a warp scheduler that can issue one instruction per clock for one of its warps, 16 FP32 lanes, 16 INT32 lanes, a 16K x 32-bit register file slice, and (on TU104 only) 2 Tensor Cores. A warp of 32 threads executes one instruction for all 32 threads together (SIMT); an FP32 instruction therefore takes 2 clocks on a 16-lane block. Latency (~4 clocks for FFMA, hundreds for DRAM loads) is hidden by switching between resident warps - so the number of warps that fit on an SM (occupancy) matters.

9.1 Occupancy, computed by hand for your kernels

Resident blocks per SM = min(register limit, thread limit, shared-memory limit, 16 block slots). Registers are allocated per warp in chunks of 256: $\text{regs/warp} = \text{ceil}(\text{regs_per_thread} \times 32 / 256) \times 256$. The results below reproduce CUPTI's 'est. achieved occupancy' column.

kernel (leg)	regs/t hr	thr/blk	smem B	regs/wa rp	blk by regs	blk by thr	blk by smem	blocks	warp s	occ %	CUPTI %	limit
_5x_cudnn_volta_scudnn_128x64_relu_xregs_large_n (windows)	160	128	16384	5120	3	8	4	3	12	38	38	registers
cudnn::bn_fw_inf_1C11_kernel_NC HW<...> (windows)	19	512	0	768	5	2	16	2	32	100	100	threads
at::native::vectorized_elementwise_kernel<4, ... (windows)	28	128	0	1024	16	8	16	8	32	100	100	threads
at::native::GLOBAL_N_71d79c19_19_DilatedMaxPo (windows)	24	256	0	768	10	4	16	4	32	100	100	threads
cudnn::winograd::generateWinogradTilesKernel<0, (windows)	40	128	8704	1280	12	8	7	7	28	88	25	shared memory
_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg (windows)	126	256	49152	4096	2	4	1	1	8	25	25	shared memory
_5x_cudnn_volta_scudnn_128x128_relu_medium_nn_v1 (windows)	128	256	32768	4096	2	4	2	2	16	50	50	registers
_5x_cudnn_volta_scudnn_128x32_sliced1x4_ldg4_rel (windows)	128	256	41472	4096	2	4	1	1	8	25	25	shared memory
cudnn::winograd_nonfused::winogradForwardData4x4 (windows)	64	256	22656	2048	4	4	2	2	16	50	50	shared memory
cudnn::winograd_nonfused::winogradForwardFilter4 (windows)	32	256	9216	1024	8	4	7	4	32	100	100	threads
volta_sgemm_32x128_nn (windows)	57	256	16384	2048	4	4	4	4	32	100	100	registers
cudnn::winograd_nonfused::winogradForwardOutput4 (windows)	48	256	16640	1536	5	4	3	3	24	75	75	shared memory
_5x_cudnn_volta_scudnn_128x64_relu_small_nn_v1 (windows)	128	128	16384	4096	4	8	4	4	16	50	41	registers
volta_sgemm_128x32_sliced1x4_tn (windows)	134	256	43008	4352	1	4	1	1	8	25	25	registers

Example: volta_sgemm_128x32_sliced1x4_tn uses 134 registers/thread and 256 threads. $134 \times 32 = 4288 \rightarrow 4352$ registers per warp x 8 warps = 34,816 registers per block; $65,536 / 34,816 = 1$ block per SM = 8 of 32 warps = 25%.

CUPTI reports exactly 25%. Low occupancy is a deliberate trade here: GEMM kernels keep a 128x32 output tile in registers to reuse every loaded value many times.

9.2 Reading kernel names

name fragment	meaning
volta_sgemm_128x32_sliced1x4_tn	cuBLAS single-precision GEMM from the Volta-era kernel family (runs on Turing), each block computes a 128x32 tile of C, K dimension sliced 1x4 across warps, A transposed / B not ('tn')
cublasLt::splitKreduce_kernel	the K dimension was split across blocks; this kernel sums the partial results (used when M x N is small but K is large)
scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt	cuDNN fp32 convolution with the Winograd transform (fewer multiplies for 3x3 filters), 128x128 tile, 1- and 4-wide global loads, ReLU fused into the epilogue
scudnn_128x64_relu_xregs_large_nn	implicit-GEMM convolution (im2col done on the fly), 128x64 tile, extra registers, 'large' variant
winograd::generateWinogradTilesKernel	pre-transforms the filter weights into the Winograd domain
bn_fw_inf_1C11_kernel_NCHW	BatchNorm forward in inference mode (per-channel scale+shift), NCHW layout
vectorized_elementwise_kernel<4, ...>	ATen's generic pointwise kernel; <4> = each thread handles 4 elements with 128-bit vector loads (ReLU, add, etc.)
max_pool_forward_nchw	ATen max-pool, one thread per output element
computeOffsetsKernel (cask)	cuDNN helper that precomputes address offsets for the implicit-GEMM kernel

10. Every kernel your models launch

Captured with torch.profiler (Kineto + CUPTI activity API): ew_classifier on a batch of 256 signals and ResNet18 on 8 images, including the input copy host->device and the result copy device->host. grid and block are the CUDA launch dimensions; regs = registers per thread from the cubin; smem = static+dynamic shared memory per block; occ = CUPTI's estimated occupancy.

10.1 ew_classifier - Windows 11 native: 20 kernels, 1,220 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	80.6	2x8x1	256x1x1	134	43008	25	volta_sgemm_128x32_sliced1x4_tn
1	8.2	1x1x1	128x1x1	26	0	1	at::native::vectorized_elementwise_kernel<4, ...>
2	18.8	8x1x1	32x16x1	36	0	25	at::native::batch_norm_transform_input_channels_last_kernel<...>
3	11.2	64x1x1	128x1x1	28	0	50	at::native::vectorized_elementwise_kernel<4, ...>
4	344.8	4x8x4	256x1x1	57	16384	100	volta_sgemm_128x32_tn
5	9.0	1x1x1	128x1x1	26	0	1	at::native::vectorized_elementwise_kernel<4, ...>
6	80.3	16x1x1	32x16x1	36	0	50	at::native::batch_norm_transform_input_channels_last_kernel<...>
7	31.1	128x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
8	336.4	8x8x2	128x1x1	86	32768	25	volta_sgemm_32x32_sliced1x4_tn
9	62.8	8x16x1	32x16x1	44	0	100	cublasLt::splitKreduce_kernel<32, ...>
10	8.4	1x1x1	128x1x1	26	0	1	at::native::vectorized_elementwise_kernel<4, ...>
11	16.6	8x1x1	32x16x1	36	0	25	at::native::batch_norm_transform_input_channels_last_kernel<...>
12	11.1	64x1x1	128x1x1	28	0	50	at::native::vectorized_elementwise_kernel<4, ...>
13	89.3	4x8x1	128x1x1	86	32768	25	volta_sgemm_32x32_sliced1x4_tn
14	7.6	1x1x1	128x1x1	26	0	1	at::native::vectorized_elementwise_kernel<4, ...>
15	22.6	4x1x1	32x16x1	36	0	13	at::native::batch_norm_transform_input_channels_last_kernel<...>
16	8.0	32x1x1	128x1x1	28	0	25	at::native::vectorized_elementwise_kernel<4, ...>
17	42.1	2x8x1	128x1x1	86	32768	13	volta_sgemm_32x32_sliced1x4_tn
18	6.6	16x1x1	128x1x1	28	0	13	at::native::vectorized_elementwise_kernel<4, ...>
19	24.6	1x8x1	128x1x1	86	32768	6	volta_sgemm_32x32_sliced1x4_tn
	45.6						Memcpy HtoD (Pageable -> Device) 131072 B @ 2.87 GB/s
	6.7						Memcpy DtoD (Device -> Device) 1024 B @ 0.15 GB/s
	2.5						Memset (Device) 128 B @ 0.05 GB/s
	9.5						Memcpy DtoD (Device -> Device) 2048 B @ 0.22 GB/s
	6.5						Memcpy DtoD (Device -> Device) 1024 B @ 0.16 GB/s
	6.4						Memcpy DtoD (Device -> Device) 512 B @ 0.08 GB/s
	5.7						Memcpy DtoH (Device -> Pageable) 8192 B @ 1.45 GB/s

CPU-side CUDA API calls: cudaStreamIsCapturing x2, cudaMemcpyAsync x6, cudaStreamSynchronize x2, cudaLaunchKernel x20, cudaMemcpyAsync x1, cudaOccupancyMaxActiveBlocksPerMultiprocessor x2, cudaDeviceSynchronize x2

10.1 ew_classifier - WSL2 / Docker (Linux): 20 kernels, 1,224 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	82.2	2x8x1	256x1x1	134	43008	25	volta_sgemm_128x32_sliced1x4_tn
1	7.4	1x1x1	128x1x1	20	0	1	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
2	18.7	8x1x1	32x16x1	36	0	25	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>{
3	9.5	128x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
4	344.2	4x8x4	256x1x1	57	16384	100	volta_sgemm_128x32_tn
5	7.0	1x1x1	128x1x1	20	0	1	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
6	88.2	16x1x1	32x16x1	36	0	50	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>{
7	27.7	256x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
8	325.8	8x8x2	128x1x1	86	32768	25	volta_sgemm_32x32_sliced1x4_tn
9	57.9	8x16x1	32x16x1	44	0	100	cutblat::splitKreduce_kernel<32, 16, int, float, float, float, false, float, float,
10	7.5	1x1x1	128x1x1	20	0	1	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
11	35.6	8x1x1	32x16x1	36	0	25	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>{
12	9.6	128x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
13	88.4	4x8x1	128x1x1	86	32768	25	volta_sgemm_32x32_sliced1x4_tn
14	6.7	1x1x1	128x1x1	20	0	1	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
15	24.3	4x1x1	32x16x1	36	0	13	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>{
16	7.2	64x1x1	128x1x1	22	0	50	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
17	44.8	2x8x1	128x1x1	86	32768	13	volta_sgemm_32x32_sliced1x4_tn
18	6.2	32x1x1	128x1x1	22	0	25	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
19	24.9	1x8x1	128x1x1	86	32768	6	volta_sgemm_32x32_sliced1x4_tn
	44.1						Memcpy HtoD (Pageable -> Device) 131072 B @ 2.97 GB/s
	6.4						Memcpy DtoD (Device -> Device) 1024 B @ 0.16 GB/s
	0.8						Memset (Device) 128 B @ 0.15 GB/s
	9.3						Memcpy DtoD (Device -> Device) 2048 B @ 0.22 GB/s
	9.4						Memcpy DtoD (Device -> Device) 1024 B @ 0.11 GB/s
	6.3						Memcpy DtoD (Device -> Device) 512 B @ 0.08 GB/s
	5.0						Memcpy DtoH (Device -> Pageable) 8192 B @ 1.65 GB/s

CPU-side CUDA API calls: cudaMemcpyAsync x6, cudaStreamSynchronize x2, cudaLaunchKernel x20, cudaMemcpyAsync x1, cudaMemcpyMaxActiveBlocksPerMultiprocessor x2, cudaDeviceSynchronize x2

10.1 ew_classifier - AWS g4dn.xlarge: 21 kernels, 113 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	10.7	8x8x1	128x1x1	86	32768	20	volta_sgemv_32x32_sliced1x4_tn
1	1.5	1x1x1	128x1x1	20	0	0	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
2	3.6	8x1x1	32x16x1	36	0	10	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>()
3	1.6	128x1x1	128x1x1	22	0	40	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
4	27.8	4x4x4	128x1x1	122	12800	20	volta_sgemv_128x64_tn
5	1.5	1x1x1	128x1x1	20	0	0	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
6	3.8	16x1x1	32x16x1	36	0	20	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>()
7	2.0	256x1x1	128x1x1	22	0	80	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
8	18.5	2x4x4	128x1x1	122	12800	10	volta_sgemv_128x64_tn
9	3.8	8x16x1	32x16x1	44	0	100	cutensor::splitKreduce_kernel<32, 16, int, float, float, float, false, float, float, float>
10	1.5	1x1x1	128x1x1	20	0	0	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
11	3.7	8x1x1	32x16x1	36	0	10	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>()
12	1.6	128x1x1	128x1x1	22	0	40	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
13	8.6	4x2x4	256x1x1	57	16384	20	volta_sgemv_32x128_tn
14	2.5	4x16x1	32x16x1	44	0	80	cutensor::splitKreduce_kernel<32, 16, int, float, float, float, false, float, float, float>
15	1.5	1x1x1	128x1x1	20	0	0	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::batc
16	3.6	4x1x1	32x16x1	36	0	5	at::native::batch_norm_transform_input_channels_last_kernel<float, float, float, 4>()
17	1.4	64x1x1	128x1x1	22	0	20	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
18	7.2	2x8x1	128x1x1	86	32768	5	volta_sgemv_32x32_sliced1x4_tn
19	1.3	32x1x1	128x1x1	22	0	10	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
20	4.9	1x8x1	128x1x1	86	32768	3	volta_sgemv_32x32_sliced1x4_tn
	22.5						Memcpy HtoD (Pageable -> Device) 131072 B @ 5.82 GB/s
	1.7						Memcpy DtoD (Device -> Device) 1024 B @ 0.62 GB/s
	0.4						Memset (Device) 64 B @ 0.18 GB/s
	1.3						Memcpy DtoD (Device -> Device) 2048 B @ 1.52 GB/s
	1.5						Memcpy DtoD (Device -> Device) 1024 B @ 0.68 GB/s
	1.3						Memcpy DtoD (Device -> Device) 512 B @ 0.38 GB/s
	1.5						Memcpy DtoH (Device -> Pageable) 8192 B @ 5.57 GB/s

CPU-side CUDA API calls: cudaMemcpyAsync x6, cudaStreamSynchronize x2, cudaOccupancyMaxActiveBlocksPerMultiprocessor x4, cudaLaunchKernel x21, cudaMemcpyAsync x1, cudaDeviceSynchronize x2

10.2 resnet18 - Windows 11 native: 103 kernels, 20,279 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	2.2	50x1x1	256x1x1	16	0	78	cask__5x_cudnn::computeOffsetsKernel<...>
1	830.0	784x1x1	128x1x1	160	16384	38	_5x_cudnn_volta_scudnn_128x64_relu_xregs_large_nn_v1
2	545.8	64x8x1	512x1x1	19	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<...>
3	531.5	6272x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
4	474.1	6272x1x1	256x1x1	24	0	100	at::native::_GLOBAL__N__71d79c19_19_DilatedMaxPool2d_cu_9872839c::max_pool_forward_n
5	5.1	2x16x1	32x4x1	40	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, ...>
6	665.6	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
7	135.6	64x8x1	512x1x1	19	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<...>
8	133.7	1568x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
9	5.2	2x16x1	32x4x1	40	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, ...>
10	666.9	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
11	135.7	64x8x1	512x1x1	19	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<...>
12	199.1	1568x1x1	128x1x1	44	0	100	at::native::vectorized_elementwise_kernel<4, ...>
13	133.1	1568x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
14	5.1	2x16x1	32x4x1	40	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, ...>
15	663.2	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
16	135.1	64x8x1	512x1x1	19	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<...>
17	133.4	1568x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
18	5.1	2x16x1	32x4x1	40	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, ...>
19	666.5	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
20	135.9	64x8x1	512x1x1	19	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<...>
21	198.8	1568x1x1	128x1x1	44	0	100	at::native::vectorized_elementwise_kernel<4, ...>
22	132.8	1568x1x1	128x1x1	28	0	100	at::native::vectorized_elementwise_kernel<4, ...>
23	2.0	4x1x1	256x1x1	16	0	6	cask__5x_cudnn::computeOffsetsKernel<...>
	1713.6						Memcpy HtoD (Pageable -> Device) 4816896 B @ 2.81 GB/s
	5.7						Memcpy DtoH (Device -> Pageable) 32000 B @ 5.65 GB/s

CPU-side CUDA API calls: cudaStreamIsCapturing x2, cudaMemcpyAsync x2, cudaStreamSynchronize x2, cudaLaunchKernel x103, cudaPeekAtLastError x28, cudaMemcpyAsync x7, cudaDeviceSynchronize x2

10.2 resnet18 - WSL2 / Docker (Linux): 109 kernels, 19,573 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	2.1	50x1x1	256x1x1	16	0	78	cask__5x_cudnn::computeOffsetsKernel<false, false>(cask__5x_cudnn::ComputeOffsetsPar
1	817.5	784x1x1	128x1x1	128	16384	50	_5x_cudnn_volta_scudnn_128x64_relu_medium_nn_v1
2	471.9	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
3	460.0	12544x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
4	411.7	6272x1x1	256x1x1	25	0	100	at::native::(anonymous namespace)::max_pool_forward_nchw<float>(int, float const*, 1
5	4.6	2x16x1	32x4x1	56	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
6	645.3	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
7	117.7	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
8	115.9	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
9	4.5	2x16x1	32x4x1	56	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
10	652.4	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
11	118.0	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
12	172.3	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunction_add<float>, std
13	115.4	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
14	4.4	2x16x1	32x4x1	56	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
15	649.8	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
16	117.9	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
17	115.3	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
18	4.6	2x16x1	32x4x1	56	8704	25	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
19	651.0	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
20	117.4	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
21	172.4	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunction_add<float>, std
22	115.4	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
23	1.9	4x1x1	256x1x1	16	0	6	cask__5x_cudnn::computeOffsetsKernel<false, false>(cask__5x_cudnn::ComputeOffsetsPar
	49129.1						Memcpy HtoD (Pageable -> Device) 4816896 B @ 0.10 GB/s
	4.4						Memcpy DtoH (Device -> Pageable) 32000 B @ 7.19 GB/s

CPU-side CUDA API calls: cudaMemcpyAsync x2, cudaStreamSynchronize x2, cudaLaunchKernel x109, cudaPeekAtLastError x22, cudaMemcpyAsync x7, cudaDeviceSynchronize x2

10.2 resnet18 - AWS g4dn.xlarge: 109 kernels, 7,872 us of GPU time

#	us	grid	block	regs	smem	occ	kernel
0	1.9	50x1x1	256x1x1	16	0	31	cask__5x_cudnn::computeOffsetsKernel<false, false>(cask__5x_cudnn::ComputeOffsetsPar
1	304.8	784x1x1	128x1x1	128	16384	50	_5x_cudnn_volta_scudnn_128x64_relu_medium_nn_v1
2	230.0	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
3	216.3	12544x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
4	194.4	6272x1x1	256x1x1	25	0	100	at::native::(anonymous namespace)::max_pool_forward_nchw<float>(int, float const*, 1
5	3.1	2x16x1	32x4x1	56	8704	10	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
6	239.6	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
7	57.3	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
8	52.8	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
9	3.7	2x16x1	32x4x1	56	8704	10	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
10	245.4	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
11	57.3	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
12	81.2	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunction_add<float>, std
13	53.9	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
14	3.4	2x16x1	32x4x1	56	8704	10	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
15	245.3	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
16	57.2	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
17	54.8	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
18	3.3	2x16x1	32x4x1	56	8704	10	cudnn::winograd::generateWinogradTilesKernel<0, float, float>(cudnn::winograd::Gener
19	243.6	16x7x4	256x1x1	126	49152	25	_5x_cudnn_volta_scudnn_winograd_128x128_ldg1_ldg4_relu_tile148t_nt_v1
20	57.0	64x8x1	512x1x1	16	0	100	cudnn::bn_fw_inf_1C11_kernel_NCHW<float, float, true, 1>(float, float, cudnnTensorSt
21	81.0	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::CUDAFunction_add<float>, std
22	54.9	3136x1x1	128x1x1	22	0	100	at::native::vectorized_elementwise_kernel<4, at::native::(anonymous namespace)::laun
23	2.2	4x1x1	256x1x1	16	0	3	cask__5x_cudnn::computeOffsetsKernel<false, false>(cask__5x_cudnn::ComputeOffsetsPar
	944.0						Memcpy HtoD (Pageable -> Device) 4816896 B @ 5.10 GB/s
	5.7						Memcpy DtoH (Device -> Pageable) 32000 B @ 5.59 GB/s

CPU-side CUDA API calls: cudaMemcpyAsync x2, cudaStreamSynchronize x2, cudaLaunchKernel x109, cudaPeekAtLastError x22, cudaMemcpyAsync x7, cudaDeviceSynchronize x2

Observe: (1) a 3-layer MLP on 256 samples = 20 launches but only ~0.1 ms of GPU work on the T4 - launch/dispatch overhead dominates; (2) ResNet18 is ~100 kernels, dominated by Winograd and implicit-GEMM convolutions; (3) the input copy for 8 images (4.8 MB) takes longer than several convolutions - the PCIe bus is a first-class cost for the image and detection models.

11. Machine code: Triton -> TTIR -> LLVM -> PTX -> SASS

cuBLAS/cuDNN kernels are shipped as pre-compiled binaries, so to see a kernel's complete life we compile one ourselves - a fused bias-add + ReLU, the same epilogue every Linear/Conv layer of these models ends with - using Triton (bundled with PyTorch on Linux):

```
@triton.jit
def bias_relu(x_ptr, b_ptr, y_ptr, n, C: tl.constexpr, BLOCK: tl.constexpr):
    pid = tl.program_id(0) # blockIdx.x
    offs = pid * BLOCK + tl.arange(0, BLOCK) # this block's 1024 element indices
    mask = offs < n # bounds check -> predicate registers
    x = tl.load(x_ptr + offs, mask=mask) # global -> registers (LDG)
    b = tl.load(b_ptr + offs % C, mask=mask) # broadcast bias (offs mod 64)
    y = tl.maximum(x + b, 0.0) # FADD + FMNMX
    tl.store(y_ptr + offs, y, mask=mask) # registers -> global (STG)
grid = (n // 1024,) n = 16384, C = 64
```

Captured on WSL2 / Docker (Linux): 26 registers/thread, 0 spills, 0 B shared memory, num_warps = 4 (= 128 threads per block, each thread owns 8 elements).

11.1 Triton IR (TTIR) - target-independent, tensor-level

```
#loc = loc("/app/benchmark/lowlevel_trace.py":405:0)
module {
  tt.func public @bias_relu(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0), %arg1: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0), %arg2: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0), %arg3: i32 {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0)) attributes {noinline = false} {
    %cst = arith.constant dense<0.000000e+00> : tensor<1024xf32> loc(#loc1)
    %cst_0 = arith.constant dense<64> : tensor<1024xi32> loc(#loc1)
    %c1024_i32 = arith.constant 1024 : i32 loc(#loc1)
    %0 = tt.get_program_id x : i32 loc(#loc2)
    %1 = arith.muli %0, %c1024_i32 : i32 loc(#loc3)
    %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32> loc(#loc4)
    %3 = tt.splat %1 : i32 -> tensor<1024xi32> loc(#loc5)
    %4 = arith.addi %3, %2 : tensor<1024xi32> loc(#loc5)
    %5 = tt.splat %arg3 : i32 -> tensor<1024xi32> loc(#loc6)
    %6 = arith.cmpi slt, %4, %5 : tensor<1024xi32> loc(#loc6)
    %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>> loc(#loc7)
    %8 = tt.addptr %7, %4 : tensor<1024x!tt.ptr<f32>>, tensor<1024xi32> loc(#loc7)
    %9 = tt.load %8, %6 : tensor<1024x!tt.ptr<f32>> loc(#loc8)
    %10 = arith.remsi %4, %cst_0 : tensor<1024xi32> loc(#loc9)
    %11 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>> loc(#loc10)
    %12 = tt.addptr %11, %10 : tensor<1024x!tt.ptr<f32>>, tensor<1024xi32> loc(#loc10)
    %13 = tt.load %12, %6 : tensor<1024x!tt.ptr<f32>> loc(#loc11)
    %14 = arith.addf %9, %13 : tensor<1024xf32> loc(#loc12)
    %15 = arith.maxnumf %14, %cst : tensor<1024xf32> loc(#loc13)
    %16 = tt.splat %arg2 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>> loc(#loc14)
    %17 = tt.addptr %16, %4 : tensor<1024x!tt.ptr<f32>>, tensor<1024xi32> loc(#loc14)
    tt.store %17, %15, %6 : tensor<1024x!tt.ptr<f32>> loc(#loc15)
    tt.return loc(#loc16)
  } loc(#loc)
} loc(#loc)
#loc1 = loc(unknown)
#loc2 = loc("/app/benchmark/lowlevel_trace.py":406:24)
#loc3 = loc("/app/benchmark/lowlevel_trace.py":407:17)
#loc4 = loc("/app/benchmark/lowlevel_trace.py":407:38)
#loc5 = loc("/app/benchmark/lowlevel_trace.py":407:25)
#loc6 = loc("/app/benchmark/lowlevel_trace.py":408:18)
#loc7 = loc("/app/benchmark/lowlevel_trace.py":409:24)
#loc8 = loc("/app/benchmark/lowlevel_trace.py":409:16)
#loc9 = loc("/app/benchmark/lowlevel_trace.py":410:31)
#loc10 = loc("/app/benchmark/lowlevel_trace.py":410:24)
#loc11 = loc("/app/benchmark/lowlevel_trace.py":410:16)
#loc12 = loc("/app/benchmark/lowlevel_trace.py":411:23)
```

11.2 Triton GPU IR (TTGIR) - adds a layout: how the 1024-element tensor is distributed over threads

```
#blocked = #triton_gpu.blocked<{sizePerThread = [4], threadsPerWarp = [32], warpsPerCTA = [4], order = [0]}>
#loc = loc("/app/benchmark/lowlevel_trace.py":405:0)
module attributes {"triton_gpu.num-ctas" = 1 : i32, "triton_gpu.num-warps" = 4 : i32, triton_gpu.target = "cuda:75", "
  triton_gpu.threads-per-warp" = 32 : i32} {
  tt.func public @bias_relu(%arg0: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":40
5:0), %arg1: !tt.ptr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0), %arg2: !tt.p
tr<f32> {tt.divisibility = 16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0), %arg3: i32 {tt.divisibility =
16 : i32} loc("/app/benchmark/lowlevel_trace.py":405:0)) attributes {noinline = false} {
  %cst = arith.constant dense<0.000000e+00> : tensor<1024xf32, #blocked> loc(#loc1)
  %c1024_i32 = arith.constant 1024 : i32 loc(#loc1)
  %cst_0 = arith.constant dense<64> : tensor<1024xi32, #blocked> loc(#loc1)
  %0 = tt.get_program_id x : i32 loc(#loc2)
  %1 = arith.muli %0, %c1024_i32 : i32 loc(#loc3)
  %2 = tt.make_range {end = 1024 : i32, start = 0 : i32} : tensor<1024xi32, #blocked> loc(#loc4)
  %3 = tt.splat %1 : i32 -> tensor<1024xi32, #blocked> loc(#loc5)
  %4 = arith.addi %3, %2 : tensor<1024xi32, #blocked> loc(#loc5)
  %5 = tt.splat %arg3 : i32 -> tensor<1024xi32, #blocked> loc(#loc6)
  %6 = arith.cmpi slt, %4, %5 : tensor<1024xi32, #blocked> loc(#loc6)
  %7 = tt.splat %arg0 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc7)
  %8 = tt.addptr %7, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc7)
  %9 = tt.load %8, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc8)
  %10 = arith.remsi %4, %cst_0 : tensor<1024xi32, #blocked> loc(#loc9)
  %11 = tt.splat %arg1 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc10)
  %12 = tt.addptr %11, %10 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc10)
  %13 = tt.load %12, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc11)
  %14 = arith.addf %9, %13 : tensor<1024xf32, #blocked> loc(#loc12)
  %15 = arith.maxnumf %14, %cst : tensor<1024xf32, #blocked> loc(#loc13)
  %16 = tt.splat %arg2 : !tt.ptr<f32> -> tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc14)
  %17 = tt.addptr %16, %4 : tensor<1024x!tt.ptr<f32>, #blocked>, tensor<1024xi32, #blocked> loc(#loc14)
  tt.store %17, %15, %6 : tensor<1024x!tt.ptr<f32>, #blocked> loc(#loc15)
  tt.return loc(#loc16)
} loc(#loc)
} loc(#loc)
#loc1 = loc(unknown)
```

The #blocked layout line is the key decision: sizePerThread=[4] means each thread holds 4 consecutive floats (-> one 128-bit load), threadsPerWarp=[32], warpsPerCTA=[num_warps]. 4 x 32 x warps x repetitions = 1024 elements.

11.3 LLVM IR (NVPTX target)

```
; ModuleID = 'LLVMDialectModule'
source_filename = "LLVMDialectModule"
target datalayout = "e-i64:64-i128:128-v16:16-v32:32-n16:32:64"

define void @bias_relu(ptr addrspace(1) %0, ptr addrspace(1) %1, ptr addrspace(1) %2, i32 %3) local_unnamed_addr !dbg
!7 {
  %5 = tail call i32 @asm "mov.u32 $0, %ctaid.x;", "=r"() #1, !dbg !10
  %6 = shl i32 %5, 10, !dbg !11
  %7 = tail call i32 @llvm.nvvm.read.ptx.sreg.tid.x(), !dbg !12
  %8 = shl i32 %7, 2, !dbg !12
  %9 = and i32 %8, 508, !dbg !12
  %10 = or disjoint i32 %6, %9, !dbg !13
  %11 = or disjoint i32 %10, 512, !dbg !13
  %12 = icmp slt i32 %10, %3, !dbg !14
  %13 = icmp slt i32 %11, %3, !dbg !14
  %14 = sext i32 %10 to i64, !dbg !15
  %15 = getelementptr float, ptr addrspace(1) %0, i64 %14, !dbg !15
  %16 = sext i32 %11 to i64, !dbg !15
  %17 = getelementptr float, ptr addrspace(1) %0, i64 %16, !dbg !15
  %18 = tail call { i32, i32, i32, i32 } @asm sideeffect "mov.u32 $0, 0x0;\0A\09mov.u32 $1, 0x0;\0A\09mov.u32 $2, 0x0;\0A\09mov.u32 $3, 0x0;\0A\09@5 ld.global.v4.b32 { $0, $1, $2, $3 }, [ $4 + 0 ];", "=r,r,r,r,l,b"(ptr addrspace(1) %15, i1 %12) #1, !dbg !16
  %19 = extractvalue { i32, i32, i32, i32 } %18, 0, !dbg !16
  %20 = extractvalue { i32, i32, i32, i32 } %18, 1, !dbg !16
  %21 = extractvalue { i32, i32, i32, i32 } %18, 2, !dbg !16
  %22 = extractvalue { i32, i32, i32, i32 } %18, 3, !dbg !16
  %23 = bitcast i32 %19 to float, !dbg !16
  %24 = bitcast i32 %20 to float, !dbg !16
  %25 = bitcast i32 %21 to float, !dbg !16
  %26 = bitcast i32 %22 to float, !dbg !16
  %27 = tail call { i32, i32, i32, i32 } @asm sideeffect "mov.u32 $0, 0x0;\0A\09mov.u32 $1, 0x0;\0A\09mov.u32 $2, 0x0;\0A\09mov.u32 $3, 0x0;\0A\09@5 ld.global.v4.b32 { $0, $1, $2, $3 }, [ $4 + 0 ];", "=r,r,r,r,l,b"(ptr addrspace(1) %17, i1 %13) #1, !dbg !16
  %28 = extractvalue { i32, i32, i32, i32 } %27, 0, !dbg !16
  %29 = extractvalue { i32, i32, i32, i32 } %27, 1, !dbg !16
  %30 = extractvalue { i32, i32, i32, i32 } %27, 2, !dbg !16
  %31 = extractvalue { i32, i32, i32, i32 } %27, 3, !dbg !16
  %32 = bitcast i32 %28 to float, !dbg !16
  %33 = bitcast i32 %29 to float, !dbg !16
  %34 = bitcast i32 %30 to float, !dbg !16
  %35 = bitcast i32 %31 to float, !dbg !16
  %36 = srem i32 %10, 64, !dbg !17
  %37 = sext i32 %36 to i64, !dbg !18
  %38 = getelementptr float, ptr addrspace(1) %1, i64 %37, !dbg !18
  %39 = tail call { i32, i32, i32, i32 } @asm sideeffect "mov.u32 $0, 0x0;\0A\09mov.u32 $1, 0x0;\0A\09mov.u32 $2, 0x0;\0A\09mov.u32 $3, 0x0;\0A\09@5 ld.global.v4.b32 { $0, $1, $2, $3 }, [ $4 + 0 ];", "=r,r,r,r,l,b"(ptr addrspace(1) %38, i1 %12) #1, !dbg !19
  %40 = extractvalue { i32, i32, i32, i32 } %39, 0, !dbg !19
  %41 = extractvalue { i32, i32, i32, i32 } %39, 1, !dbg !19
  %42 = extractvalue { i32, i32, i32, i32 } %39, 2, !dbg !19
  %43 = extractvalue { i32, i32, i32, i32 } %39, 3, !dbg !19
  %44 = bitcast i32 %40 to float, !dbg !19
  %45 = bitcast i32 %41 to float, !dbg !19
  %46 = bitcast i32 %42 to float, !dbg !19
  %47 = bitcast i32 %43 to float, !dbg !19
  %48 = tail call { i32, i32, i32, i32 } @asm sideeffect "mov.u32 $0, 0x0;\0A\09mov.u32 $1, 0x0;\0A\09mov.u32 $2, 0x0;\0A\09mov.u32 $3, 0x0;\0A\09@5 ld.global.v4.b32 { $0, $1, $2, $3 }, [ $4 + 0 ];", "=r,r,r,r,l,b"(ptr addrspace(1) %38, i1 %13) #1, !dbg !19
  %49 = extractvalue { i32, i32, i32, i32 } %48, 0, !dbg !19
  %50 = extractvalue { i32, i32, i32, i32 } %48, 1, !dbg !19
  %51 = extractvalue { i32, i32, i32, i32 } %48, 2, !dbg !19
  %52 = extractvalue { i32, i32, i32, i32 } %48, 3, !dbg !19
  %53 = bitcast i32 %49 to float, !dbg !19
  %54 = bitcast i32 %50 to float, !dbg !19
```

11.4 PTX - NVIDIA's virtual ISA (what the driver can still re-compile for any future GPU)

PTX	meaning
.visible .entry bias_relu(	Kernel entry point; .param lines are the kernel arguments (pointers and n) placed in constant bank 0 by the driver.
.param .u64 .ptr .global .align 1 bias_relu_param_0,	
.param .u64 .ptr .global .align 1 bias_relu_param_1,	
.param .u64 .ptr .global .align 1 bias_relu_param_2,	
.param .u32 bias_relu_param_3	
)	
.reqntid 128, 1, 1	
{	
.reg .pred %p<7>;	Declares virtual registers (unlimited in PTX; ptxas later allocates the physical 32-bit registers per thread).
.reg .b32 %r<38>;	Declares virtual registers (unlimited in PTX; ptxas later allocates the physical 32-bit registers per thread).
.reg .f32 %f<33>;	Declares virtual registers (unlimited in PTX; ptxas later allocates the physical 32-bit registers per thread).
.reg .b64 %rd<12>;	Declares virtual registers (unlimited in PTX; ptxas later allocates the physical 32-bit registers per thread).
ld.param.u64 %rd7, [bias_relu_param_0];	Load a kernel argument.
ld.param.u64 %rd8, [bias_relu_param_1];	Load a kernel argument.
mov.u32 %r1, %ctaid.x;	Block index (blockIdx) = Triton program_id(0).
shl.b32 %r26, %r1, 10;	Integer index arithmetic: offs = pid*BLOCK + lane offsets.
ld.param.u64 %rd9, [bias_relu_param_2];	Load a kernel argument.
ld.param.u32 %r27, [bias_relu_param_3];	Load a kernel argument.
mov.u32 %r28, %tid.x;	Thread index inside the block (threadIdx).
shl.b32 %r29, %r28, 2;	Integer index arithmetic: offs = pid*BLOCK + lane offsets.
and.b32 %r30, %r29, 508;	
or.b32 %r31, %r26, %r30;	
or.b32 %r32, %r31, 512;	
setp.lt.s32 %p1, %r31, %r27;	Compare -> predicate (%p) for the mask.
setp.lt.s32 %p2, %r32, %r27;	Compare -> predicate (%p) for the mask.
mul.wide.s32 %rd10, %r31, 4;	64-bit address computation base + 4*index.
add.s64 %rd1, %rd7, %rd10;	64-bit address computation base + 4*index.
add.s64 %rd2, %rd1, 2048;	64-bit address computation base + 4*index.
mov.u32 %r2, 0x0;	
mov.u32 %r3, 0x0;	
mov.u32 %r4, 0x0;	
mov.u32 %r5, 0x0;	
@%p1 ld.global.v4.b32 { %r2, %r3, %r4, %r5 }, [%rd1 + 0];	Vector load from global memory (v4 = 4 x 32 bit = 128-bit access), guarded by @%p predicate.
mov.b32 %f1, %r2;	
mov.b32 %f2, %r3;	
mov.b32 %f3, %r4;	
mov.b32 %f4, %r5;	
mov.u32 %r6, 0x0;	
mov.u32 %r7, 0x0;	
mov.u32 %r8, 0x0;	
mov.u32 %r9, 0x0;	
@%p2 ld.global.v4.b32 { %r6, %r7, %r8, %r9 }, [%rd2 + 0];	Vector load from global memory (v4 = 4 x 32 bit = 128-bit access), guarded by @%p predicate.
mov.b32 %f5, %r6;	
mov.b32 %f6, %r7;	
mov.b32 %f7, %r8;	

PTX	meaning
mov.b32 %f8, %r9;	
bfe.s32 %r33, %r1, 21, 1;	
shr.u32 %r34, %r33, 26;	
add.s32 %r35, %r31, %r34;	
and.b32 %r36, %r35, -64;	
sub.s32 %r37, %r31, %r36;	
mul.wide.s32 %rd11, %r37, 4;	64-bit address computation base + 4*index.
add.s64 %rd3, %rd8, %rd11;	64-bit address computation base + 4*index.
mov.u32 %r10, 0x0;	
mov.u32 %r11, 0x0;	
mov.u32 %r12, 0x0;	
mov.u32 %r13, 0x0;	
@%p1 ld.global.v4.b32 { %r10, %r11, %r12, %r13 }, [%rd3 + 0];	Vector load from global memory (v4 = 4 x 32 bit = 128-bit access), guarded by @%p predicate.
mov.b32 %f9, %r10;	
mov.b32 %f10, %r11;	
mov.b32 %f11, %r12;	
mov.b32 %f12, %r13;	
mov.u32 %r14, 0x0;	
mov.u32 %r15, 0x0;	
mov.u32 %r16, 0x0;	
mov.u32 %r17, 0x0;	
@%p2 ld.global.v4.b32 { %r14, %r15, %r16, %r17 }, [%rd3 + 0];	Vector load from global memory (v4 = 4 x 32 bit = 128-bit access), guarded by @%p predicate.
mov.b32 %f13, %r14;	
mov.b32 %f14, %r15;	
mov.b32 %f15, %r16;	

11.5 SASS - the real sm_75 machine code (cuobjdump -sass)

ptxas turns PTX into SASS: it allocates physical registers, schedules instructions, and encodes each as a 128-bit word. The /*0080*/ column is the byte offset (16 bytes per instruction); the hex column is the encoding, whose top bits hold the compiler-set scheduling control: stall cycles, yield hint, and which of the 6 scoreboard barriers the instruction sets or waits on (this is how a load's result is waited for without hardware interlocks).

offset	instruction	encoding (hex, 2 x 64 bit)	what the SM does
0000	IMAD.MOV.U32 R1, RZ, RZ, c[0x0][0x28]	0x00000a00ff017624 0x000fe400078e00ff	Integer multiply-add on the FMA-capable integer pipe; the compiler also uses it as a MOV/SHIFT/ADD because that pipe is otherwise idle.
0010	S2R R0, SR_TID.X	0x00000000000007919 0x000e220000002100	Special-register -> general register read (e.g. SR_TID.X = threadldx.x, SR_CTAID.X = blockldx.x). Goes through a slow, variable-latency path, so the scheduler tracks it with a scoreboard barrier.
0020	IMAD.MOV.U32 R19, RZ, RZ, 0x4	0x00000004ff137424 0x000fe200078e00ff	Integer multiply-add on the FMA-capable integer pipe; the compiler also uses it as a MOV/SHIFT/ADD because that pipe is otherwise idle.
0030	CS2R R12, SRZ	0x00000000000c7805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
0040	CS2R R14, SRZ	0x00000000000e7805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
0050	S2R R3, SR_CTAID.X	0x0000000000037919 0x000e620000002500	Special-register -> general register read (e.g. SR_TID.X = threadldx.x, SR_CTAID.X = blockldx.x). Goes through a slow, variable-latency path, so the scheduler tracks it with a scoreboard barrier.
0060	CS2R R20, SRZ	0x00000000000147805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
0070	CS2R R22, SRZ	0x00000000000167805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
0080	CS2R R4, SRZ	0x0000000000047805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
0090	CS2R R6, SRZ	0x0000000000067805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
00a0	CS2R R8, SRZ	0x0000000000087805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
00b0	CS2R R10, SRZ	0x00000000000a7805 0x000fe2000001ff00	Fast read of a special register pair (e.g. SRZ = zero, clock).
00c0	IMAD.SHL.U32 R0, R0, 0x4, RZ	0x0000000400007824 0x001fca00078e00ff	Integer multiply-add on the FMA-capable integer pipe; the compiler also uses it as a MOV/SHIFT/ADD because that pipe is otherwise idle.
00d0	LOP3.LUT R0, R0, 0x1fc, RZ, 0xc0, !PT	0x000001fc00007812 0x000fca00078ec0ff	Arbitrary 3-input bitwise function selected by an 8-bit truth-table immediate (0xc0 = a AND b). With 0x1fc it masks lane offsets; with 0xfffffc0 (= ~63) it rounds down to a multiple of 64 for offs % 64.
00e0	IMAD R0, R3, 0x400, R0	0x0000040003007824 0x002fe200078e0200	Integer multiply-add on the FMA-capable integer pipe; the compiler also uses it as a MOV/SHIFT/ADD because that pipe is otherwise idle.
00f0	SHF.R.S32.HI R3, RZ, 0x15, R3	0x00000015ff037819 0x000fc80000011403	Funnel/logical shift.
0100	SGXT R3, R3, 0x1	0x000000010303781a 0x000fe4000000200	Sign-extend a bit-field: part of the signed `offs % 64` trick (fix-up for negative offsets).
0110	IADD3 R2, R0.reuse, 0x200, RZ	0x0000020000027810 0x040fe40007ffe0ff	Three-input integer add.
0120	LEA.HI R3, R3, R0, RZ, 0x6	0x0000000003037211 0x000fe400078f30ff	Load-effective-address: (a << shift) + b in one instruction, used for index*16 byte offsets.
0130	ISETP.GE.AND P0, PT, R0, c[0x0][0x178], PT	0x000005e000007a0c 0x000fe40003f06270	Integer compare -> writes a 1-bit predicate register P0..P6 (used for the `mask = offs < n` bounds check).
0140	LOP3.LUT R3, R3, 0xfffffc0, RZ, 0xc0, !PT	0xfffffc0030307812 0x000fe400078ec0ff	Arbitrary 3-input bitwise function selected by an 8-bit truth-table immediate (0xc0 = a AND b). With 0x1fc it masks lane offsets; with 0xfffffc0 (= ~63) it rounds down to a multiple of 64 for offs % 64.
0150	ISETP.GE.AND P1, PT, R2, c[0x0][0x178], PT	0x000005e002007a0c 0x000fc60003f26270	Integer compare -> writes a 1-bit predicate register P0..P6 (used for the `mask = offs < n` bounds check).

offset	instruction	encoding (hex, 2 x 64 bit)	what the SM does
0160	IMAD.IADD R16, R0.reuse, 0x1, -R3	0x0000000100107824 0x040fe400078e0a03	Integer multiply-add on the FMA-capable integer pipe; the compiler also uses it as a MOV/SHIFT/ADD because that pipe is otherwise idle.
0170	IMAD.WIDE R2, R0, R19, c[0x0][0x160]	0x0000580000027625 0x000fc800078e0213	32x32 -> 64-bit integer multiply-add: pointer arithmetic base + index*4 (sizeof(float)) into a 64-bit address register pair Rn:Rn+1.
0180	IMAD.WIDE R16, R16, R19, c[0x0][0x168]	0x00005a0010107625 0x000fc800078e0213	32x32 -> 64-bit integer multiply-add: pointer arithmetic base + index*4 (sizeof(float)) into a 64-bit address register pair Rn:Rn+1.
0190	@!P0 LDG.E.128.SYS R12, [R2]	0x00000000020c8381 0x000ea800001eed00	Predicated on @!P0: only lanes whose predicate is true execute. Global-memory LOAD of 128 bits (4 x fp32) per thread in one instruction: a warp issues 32 x 16 B = 512 B, i.e. four fully-coalesced 128-byte L2 sectors-lines.
01a0	@!P0 LDG.E.128.SYS R20, [R16]	0x0000000010148381 0x000ea800001eed00	Predicated on @!P0: only lanes whose predicate is true execute. Global-memory LOAD of 128 bits (4 x fp32) per thread in one instruction: a warp issues 32 x 16 B = 512 B, i.e. four fully-coalesced 128-byte L2 sectors-lines.
01b0	@!P1 LDG.E.128.SYS R8, [R2+0x800]	0x0008000002089381 0x000ee800001eed00	Predicated on @!P1: only lanes whose predicate is true execute. Global-memory LOAD of 128 bits (4 x fp32) per thread in one instruction: a warp issues 32 x 16 B = 512 B, i.e. four fully-coalesced 128-byte L2 sectors-lines.
01c0	@!P1 LDG.E.128.SYS R4, [R16]	0x0000000010049381 0x000ee200001eed00	Predicated on @!P1: only lanes whose predicate is true execute. Global-memory LOAD of 128 bits (4 x fp32) per thread in one instruction: a warp issues 32 x 16 B = 512 B, i.e. four fully-coalesced 128-byte L2 sectors-lines.
01d0	FADD R12, R20, R12	0x0000000c140c7221 0x004fc40000000000	fp32 add on the FMA pipe (x + b).
01e0	FADD R13, R21, R13	0x0000000d150d7221 0x000fe40000000000	fp32 add on the FMA pipe (x + b).
01f0	FADD R14, R22, R14	0x0000000e160e7221 0x000fe20000000000	fp32 add on the FMA pipe (x + b).
0200	FMNMX R12, RZ, R12, !PT	0x0000000cff0c7209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
0210	FADD R15, R23, R15	0x0000000f170f7221 0x000fe20000000000	fp32 add on the FMA pipe (x + b).
0220	FMNMX R13, RZ, R13, !PT	0x0000000dff0d7209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
0230	FADD R8, R4, R8	0x0000000804087221 0x008fe20000000000	fp32 add on the FMA pipe (x + b).
0240	FMNMX R14, RZ, R14, !PT	0x0000000eff0e7209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
0250	FADD R9, R5, R9	0x0000000905097221 0x000fe20000000000	fp32 add on the FMA pipe (x + b).
0260	FMNMX R15, RZ, R15, !PT	0x0000000fff0f7209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
0270	IMAD.WIDE R4, R0, R19, c[0x0][0x170]	0x00005c0000047625 0x000fe200078e0213	32x32 -> 64-bit integer multiply-add: pointer arithmetic base + index*4 (sizeof(float)) into a 64-bit address register pair Rn:Rn+1.
0280	FMNMX R8, RZ, R8, !PT	0x00000008ff087209 0x000fc40007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
0290	FMNMX R9, RZ, R9, !PT	0x00000009ff097209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
02a0	FADD R6, R6, R10	0x0000000a06067221 0x000fe40000000000	fp32 add on the FMA pipe (x + b).
02b0	FADD R7, R7, R11	0x0000000b07077221 0x000fe40000000000	fp32 add on the FMA pipe (x + b).
02c0	@!P0 STG.E.128.SYS [R4], R12	0x0000000c04008386 0x0001e2000010ed00	Predicated on @!P0: only lanes whose predicate is true execute. Global-memory STORE of 128 bits per thread (4 floats) - the vectorised write of y.
02d0	FMNMX R10, RZ, R6, !PT	0x00000006ff0a7209 0x000fe40007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.
02e0	FMNMX R11, RZ, R7, !PT	0x00000007ff0b7209 0x000fe20007800000	fp32 min/max; with predicate !PT it is max(x, 0.0) = the ReLU.

offset	instruction	encoding (hex, 2 x 64 bit)	what the SM does
02f0	@P1 EXIT	0x000000000000194d 0x000fee0003800000	Predicated on @P1: only lanes whose predicate is true execute. Thread (warp) exits; @P0 EXIT = early-out for threads whose predicate says they are out of range.
0300	STG.E.128.SYS [R4+0x800], R8	0x0008000804007386 0x000fe2000010ed00	Global-memory STORE of 128 bits per thread (4 floats) - the vectorised write of y.
0310	EXIT	0x000000000000794d 0x000fea0003800000	Thread (warp) exits; @P0 EXIT = early-out for threads whose predicate says they are out of range.
0320	BRA 0x320	0xffffffff00007947 0x000fc0000383ffff	Branch. 'BRA' to itself after EXIT is the standard end-of-kernel padding trap.
0330	NOP	0x0000000000007918 0x000fc00000000000	Padding so the instruction stream stays aligned to 128-bit bundles / cache lines.
0340	NOP	0x0000000000007918 0x000fc00000000000	Padding so the instruction stream stays aligned to 128-bit bundles / cache lines.
0350	NOP	0x0000000000007918 0x000fc00000000000	Padding so the instruction stream stays aligned to 128-bit bundles / cache lines.
0360	NOP	0x0000000000007918 0x000fc00000000000	Padding so the instruction stream stays aligned to 128-bit bundles / cache lines.
0370	NOP	0x0000000000007918 0x000fc00000000000	Padding so the instruction stream stays aligned to 128-bit bundles / cache lines.

cuobjdump -res-usage (resource summary the driver uses to compute occupancy at launch):

```
Resource usage:
Common:
  GLOBAL:0
Function bias_relu:
  REG:26 STACK:0 SHARED:0 LOCAL:0 CONSTANT[0]:380 TEXTURE:0 SURFACE:0 SAMPLER:0
```

Trace one element through the hardware: S2R reads threadIdx/blockIdx; IMAD/LEA build the byte address; ISETP computes the mask predicate; LDG.E.128 fetches 4 floats per thread (a warp's 32 requests coalesce into 4 x 128-byte transactions from L2/DRAM); offs % 64 costs no division at all - C is a constexpr power of two, so ptxas emits SHF/SGXT/LEA.HI/LOP3 (shift, sign-fix, mask with ~63); a non-power-of-two C would have produced a MUFU.RCP + IMAD.HI reciprocal sequence because the GPU has no integer divider. FADD adds the bias, FMNMX clamps at zero, STG.E.128 writes 16 bytes per thread; EXIT ends the warp. Everything a Linear/Conv epilogue does in these models is this sequence, just fused into the cuBLAS/cuDNN kernel.

11.6 torch.compile of the repo's own ew_classifier (TorchInductor)

Inductor traces the model, keeps the matrix multiplies as cuBLAS calls (extern_kernels.mm/addmm) and fuses everything between them (bias, batch-norm, ReLU) into generated Triton kernels. Excerpt of the generated code:

```

V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code] @triton.jit
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code] def triton_poi_fused_native_batch_norm
    _legit_no_training_addmm_relu_0(in_out_ptr0, in_ptr0, in_ptr1, in_ptr2, in_ptr3, in_ptr4, xnumel, XBLOCK : tl.constexpr):
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         xnumel = 65536
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         xoffset = tl.program_id(0) * XBLOCK
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         xindex = xoffset + tl.arange(0, XBLOCK)[:XBLOCK]
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         xmask = tl.full([XBLOCK], True, tl.int1)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         x2 = xindex
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         x0 = (xindex % 256)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp0 = tl.load(in_out_ptr0 + (x2), None)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp1 = tl.load(in_ptr0 + (x0), None, eviction_policy='evict_last')
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp3 = tl.load(in_ptr1 + (x0), None, eviction_policy='evict_last')
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp5 = tl.load(in_ptr2 + (x0), None, eviction_policy='evict_last')
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp14 = tl.load(in_ptr3 + (x0), None, eviction_policy='evict_last')
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp16 = tl.load(in_ptr4 + (x0), None, eviction_policy='evict_last')
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp2 = tmp0 + tmp1
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp4 = tmp2 - tmp3
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp6 = 1e-05
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp7 = tmp5 + tmp6
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp8 = libdevice.sqrt(tmp7)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp9 = tl.full([1], 1, tl.int32)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp10 = tmp9 / tmp8
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp11 = 1.0
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp12 = tmp10 * tmp11
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp13 = tmp4 * tmp12
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp15 = tmp13 * tmp14
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp17 = tmp15 + tmp16
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         tmp18 = tl.full([1], 0, tl.int32)
...
# host-side call sequence:
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.mm(arg2_1, reinterpret_tensor(arg0_1, (128, 256), (1, 128), 0), out=buf0)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.mm(buf1, reinterpret_tensor(arg7_1, (256, 512), (1, 256), 0), out=buf2)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.mm(buf3, reinterpret_tensor(arg13_1, (512, 256), (1, 512), 0), out=buf4)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.mm(buf5, reinterpret_tensor(arg19_1, (256, 128), (1, 256), 0), out=buf6)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.mm(buf7, reinterpret_tensor(arg25_1, (128, 64), (1, 128), 0), out=buf8)
V0926 13:58:37.094000 902 torch/_inductor/graph.py:2045] [0/0] [__output_code]         extern_kernels.addmm(arg28_1, buf9, reinterpret_tensor(arg27_1, (64, 8), (1, 64), 0), alpha=1, beta=1, out=buf10)

```

12. Test results for all modes and what dominated each

Every run log was parsed for status independent of the exit code (several scripts catch Spark exceptions and still exit 0). Throughput = total samples processed by all models / wall time. Configuration per environment: Windows and WSL2 = one laptop, Spark local[2] (one concurrent task: spark.task.cpus=2), 4 GB GTX 1650; AWS = Spark standalone master + worker containers on one g4dn.xlarge, executor with 4 cores (2 concurrent tasks), 16 GB T4.

Phase 1 - mode comparison

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p1_dist_large	OK Spark 266/s 189.4 s	OK Spark 410/s 122.8 s	OK Spark 4,366/s 11.5 s
p1_modes_medium	PARTIAL - torch.load of truncated model bytes Single 19,609/s; Hybrid 6,765/s	OK Single 8,278/s; Hybrid 542/s; Spark 75/s 335.0 s	OK Single 58,608/s; Hybrid 59,092/s; Spark 2,672/s 9.4 s
p1_modes_small	PARTIAL - host RAM during CUDA JIT Single 9,681/s; Hybrid 13,745/s	OK Single 2,680/s; Hybrid 8,605/s; Spark 80/s 63.2 s	OK Single 37,162/s; Hybrid 38,178/s; Spark 561/s 9.0 s

Phase 2 - partition scaling

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p2_partitions_16	OK Spark 144/s 175.1 s	OK Spark 429/s 58.6 s	OK Spark 1,699/s 14.8 s
p2_partitions_2	OK Spark 529/s 47.6 s	OK Spark 342/s 73.6 s	OK Spark 2,454/s 10.3 s
p2_partitions_4	OK Spark 307/s 81.9 s	OK Spark 538/s 46.8 s	OK Spark 2,667/s 9.4 s
p2_partitions_8	OK Spark 204/s 123.6 s	OK Spark 356/s 70.8 s	OK Spark 2,320/s 10.8 s

Phase 3 - data size

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p3_medium	OK Spark 328/s 76.8 s	OK Spark 436/s 57.8 s	OK Spark 2,677/s 9.4 s
p3_tiny	OK Spark 36/s 70.2 s	OK Spark 34/s 75.5 s	OK Spark 292/s 8.7 s
p3_xlarge	OK Spark 396/s 126.9 s	OK Spark 574/s 87.6 s	OK Spark 4,494/s 11.2 s

Phase 4 - batch size

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p4_batch_16	OK Spark 388/s 65.0 s	OK Spark 464/s 54.3 s	OK Spark 2,569/s 9.8 s
p4_batch_256	OK Spark 318/s 79.2 s	OK Spark 514/s 49.0 s	OK Spark 2,666/s 9.4 s
p4_batch_512	OK Spark 399/s 63.1 s	OK Spark 375/s 67.2 s	OK Spark 2,645/s 9.5 s
p4_batch_64	OK Spark 421/s 59.8 s	OK Spark 325/s 77.4 s	OK Spark 2,722/s 9.2 s

Phase 6/7/10 - cluster engine device modes

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p7_cpu_only_medium	OK Spark 154/s 99.7 s	OK Spark 130/s 118.0 s	OK Spark 904/s 17.0 s
p7_cpu_only_small	OK Spark 111/s 46.7 s	OK Spark 54/s 96.8 s	OK Spark 416/s 12.5 s
p7_gpu_only_medium	OK Spark 192/s 79.9 s	OK Spark 126/s 121.8 s	OK Spark 1,444/s 10.6 s
p7_gpu_only_small	OK Spark 127/s 41.0 s	OK Spark 61/s 85.1 s	OK Spark 560/s 9.3 s
p7_hybrid_medium	OK Spark 181/s 84.9 s	OK Spark 131/s 117.2 s	OK Spark 1,449/s 10.6 s
p7_hybrid_small	OK Spark 130/s 39.9 s	OK Spark 45/s 114.5 s	OK Spark 569/s 9.1 s

Phase 8 - GPU batch size

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
p8_gpu_batch_128	FAIL - host commit limit (JVM mmap failed)	OK Spark 137/s 110.7 s	OK Spark 1,639/s 9.3 s
p8_gpu_batch_32	OK Spark 333/s 45.6 s	OK Spark 173/s 88.0 s	OK Spark 1,608/s 9.4 s
p8_gpu_batch_512	OK Spark 325/s 46.7 s	OK Spark 237/s 64.1 s	OK Spark 1,619/s 9.4 s

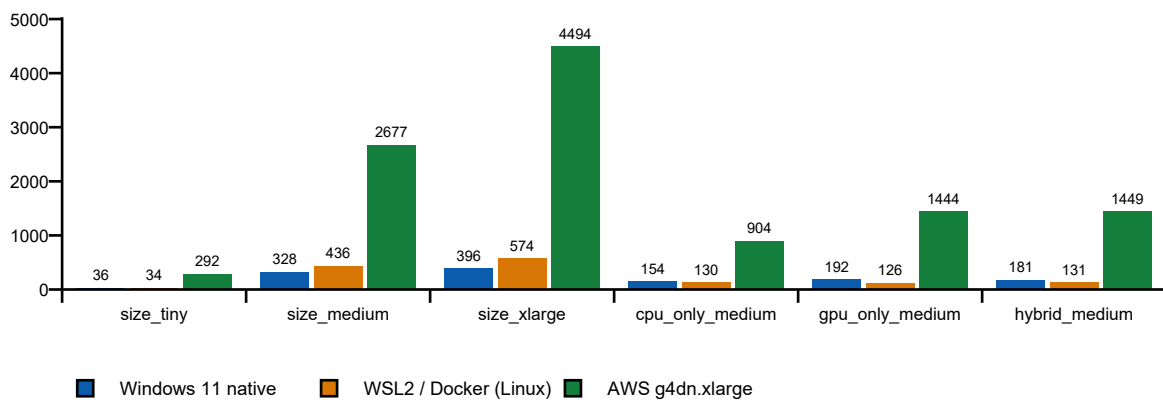
Engines - RDD vs pandas UDF (example_mlp)

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
eng_rdd_cpu_only	OK Spark 952/s 21.0 s	OK Spark 2,578/s 7.8 s	OK Spark 4,192/s 4.8 s
eng_rdd_gpu_only	OK Spark 647/s 30.9 s	OK Spark 2,376/s 8.4 s	OK Spark 3,964/s 5.0 s
eng_udf_cpu_only	OK Spark 465/s 43.0 s	OK Spark 448/s 44.6 s	OK Spark 1,844/s 10.8 s
eng_udf_gpu_only	OK Spark 405/s 49.3 s	OK Spark 974/s 20.5 s	OK Spark 1,774/s 11.3 s

Spark vs single GPU

test	Windows 11 native	WSL2 / Docker (Linux)	AWS g4dn.xlarge
svs_single_gpu	PARTIAL - host commit limit (JVM mmap failed) sequential 4,142/s; parallel_streams 3,797/s	OK sequential 1,404/s; parallel_streams 20,618/s	OK sequential 16,239/s; parallel_streams 58,842/s

Spark distributed throughput (samples/s)



12.1 What each test is really measuring at the low level

test / phase	dominant low-level effect	evidence in your data
P1 single_gpu vs distributed	Single-GPU mode keeps models resident and launches kernels on CUDA streams from one process; distributed pays per task: Python worker start (spawn on Windows), cloudpickle of 10 models' weights, model re-construction, first-call cuDNN autotune/JIT, H2D of pageable data.	Mode 1 is 10-100x slower than Mode 2 on every leg; event log shows run time >> JVM CPU time
P2 partitions	Each partition = one task = one full model load. More partitions multiply fixed overhead; parallelism is capped at 1 (laptop) or 2 (AWS) concurrent tasks.	Windows: falls monotonically 2 -> 16 (process spawn per task); WSL2 noisy (342-538/s); AWS peaks at 4 (= 2 task slots x 2 waves), falls at 16
P3 data size	Fixed per-task overhead is amortised by more samples; throughput rises with size until the GPU (or pageable PCIe copy) becomes the limit.	tiny -> xlarge: ~10-15x on every leg
P4 / P8 batch size	Larger batches = fewer kernel launches and larger GEMM tiles per launch. Once launch overhead is amortised the effect flattens; memory ceiling on 4 GB VRAM.	flat on T4 (1.6K/s at 32..512); p8 batch 128 failed on the laptop (VRAM/commit)
P7 cpu_only vs gpu_only vs hybrid	cpu_only = oneDNN AVX2/AVX-512 JIT code; gpu_only = cuBLAS/cuDNN SASS; hybrid = GPU if present. Image/detection models dominate compute.	gpu_only ~1.6x cpu_only on T4 at medium load; laptop gains less (launch latency + contention)
Engines rdd vs udf	RDD returns counts only (small result, pickle stream); UDF ships Arrow batches and returns one prediction row per sample (much larger result, JVM->Python Arrow conversion both ways).	RDD ~2x UDF throughput on every leg; UDF logs are 2.7 MB (real predictions)
svs_single_gpu	Same models, no Spark: pure CUDA streams. Upper bound for this hardware.	T4 58.8K/s; same GTX 1650: 20.6K/s under WSL2/Linux vs 3.8K/s on Windows native (WDDM launch cost)

13. Failures, root-caused to the byte

environment	test	status	root cause
Windows 11 native	p1_modes_medium	PARTIAL	torch.load of truncated model bytes
Windows 11 native	p1_modes_small	PARTIAL	host RAM during CUDA JIT
Windows 11 native	p8_gpu_batch_128	FAIL	host commit limit (JVM mmap failed)
Windows 11 native	svs_single_gpu	PARTIAL	host commit limit (JVM mmap failed)
WSL2 / Docker (Linux)	svs_single_gpu	PARTIAL	Spark half stalled (VM out of memory); single-GPU half re-run with --local

How each failure happens at the lowest level:

symptom in log	mechanism
DefaultCPUAllocator: not enough memory: you tried to allocate 589824 bytes	torch's CPU allocator asked the OS for one 128x128x3x3 fp32 ResNet conv weight (128*128*9*4 = 589,824 B; the other failure, 9,437,184 B = 512x512x3x3) and the OS refused: with local[4] four Python workers each held 10 models (~1+ GB each) on an 8 GB host with a 4 GB WSL2 VM resident.
os::commit_memory ... 'The paging file is too small' (errno 1455)	Windows commits every reserved page against RAM+pagefile at reservation time (Linux overcommits). The JVM's G1 heap tried to commit a 64 MB region; the system commit limit was exhausted, the JVM aborted (hs_err_pid*.log), Py4J lost its socket -> 'connection forcibly closed'.
fatal: Memory allocation failure / CUDA error: unknown error	The CUDA driver JIT (or cuDNN/cuBLAS lazy module load) needed host memory to load/compile kernel images and failed; the context is then poisoned and the next kernel returns cudaErrorUnknown.
CUDA error: out of memory	cudaMalloc for activations/workspace exceeded the 4 GB VRAM (minus what the Windows desktop compositor holds under WDDM).
unexpected pos X vs Y (inline_container.cc)	torch.load read a truncated serialized model (zip container) - the bytes shipped to the worker were cut short when the JVM/worker died mid-transfer.
WSL2 cluster: stage stuck (0 + 2) / 4, Docker API 500	executor processes were OOM-killed inside the 3.75 GB WSL2 VM and relaunched repeatedly; the VM itself became unresponsive. Fixed by running one container with local[2].
AWS first attempt: bucket name ""	SSM ran the script before cloud-init had written BUCKET into /etc/environment (race); fixed by passing BUCKET explicitly and 'cloud-init status --wait'.

Take-away: on an 8 GB laptop the Spark distributed modes are memory-bound, not compute-bound - each concurrent task holds its own copy of all 10 models plus a CUDA context (~300-500 MB host RSS). On AWS (16 GB host, 16 GB VRAM) every test passed.

14. Reproduce / file index

```
# Windows native leg (Spark local[2])
LEG=windows bash benchmark/run_campaign.sh
python benchmark/lowlevel_trace.py --out results/campaign_20260926/windows/lowlevel

# WSL2 / Docker leg (one GPU container, Linux)
docker run -d --name wsl-leg --gpus all -e SPARK_MASTER_URL=local[2] -v <repo>/results:/app/results ... multi-model-in
ference:latest sleep infinity
docker exec -e LEG=wsl2_docker wsl-leg bash benchmark/run_campaign.sh
docker exec wsl-leg bash -c "apt-get install -y gcc && python benchmark/lowlevel_trace.py --out results/campaign_20260
926/wsl2_docker/lowlevel"

# AWS leg (deploys GpuBenchmarkStack g4dn.xlarge, runs everything, downloads, destroys)
.\deploy\run_aws_campaign.ps1

# Summaries and this PDF
python benchmark/summarize_campaign.py results/campaign_20260926
python benchmark/build_lowlevel_pdf.py results/campaign_20260926 docs/LOWLEVEL_EXECUTION_GUIDE_20260926.pdf

# Concepts handbook (Word) - needs `npm install docx`
node benchmark/build_concepts_docx.js
```

path	contents
results/campaign_20260926/<leg>/*.log	full stdout/stderr of every test
results/campaign_20260926/<leg>/summary.tsv	exit code and wall seconds per test
results/campaign_20260926/<leg>/lowlevel.log	human-readable low-level trace (chapters 2-11)
results/campaign_20260926/<leg>/lowlevel/lowlevel.json	all trace measurements (this PDF's data source)

path	contents
results/campaign_20260926/<leg>/lowlevel/trace_*.json	Chrome/Perfetto timelines: open in ui.perfetto.dev
results/campaign_20260926/<leg>/lowlevel/bias_relu.{ttir,ttgir,llir,ptx,cubin,sass}	every compilation stage of the chapter-11 kernel
results/campaign_20260926/<leg>/spark-events/	Spark event logs (Linux legs) - load in the Spark History Server
results/campaign_20260926/aws_g4dn/spark-worker-work/	executor stdout/stderr per application
results/campaign_20260926/windows/jvm_crash/	hs_err_pid*.log / replay_pid*.log from the JVM commit-limit crashes
results/campaign_20260926/summary.md json	cross-environment status/throughput table